

Complete CA Template Function Guide

LPC1768 / Cortex-M3 - Ultimate CA Exam Template v2

5	44	90	16
answer functions	exam_* functions	precise functions	course functions

One project. One target. One answer.

Every public function is explained with parameters, return values, setup, ownership, side effects, examples and common mistakes.

Contents

Contents	2
How to use this guide	7
Status values	7
Interrupt ownership	7
Important types and constants	7
Functions you write	8
Answer structure and callbacks	8
void exam_user_init(void);	8
void exam_user_loop(void);	8
void exam_button_event(exam_button_t button, exam_button_event_t event);	8
void exam_joystick_event(uint32_t current, uint32_t changed);	9
void exam_timer_event(uint8_t timer, uint32_t flags);	9
Recommended exam_* functions	10
Program control	10
void exam_init(void);	10
void exam_idle(void);	10
LED output	10
exam_status_t exam_led_on(uint8_t printed_number);	10
exam_status_t exam_led_off(uint8_t printed_number);	11
exam_status_t exam_led_toggle(uint8_t printed_number);	11
exam_status_t exam_led_write(uint8_t mask);	11
uint8_t exam_led_read(void);	12
void exam_leds_off(void);	12
External buttons	12
exam_status_t exam_buttons_start(exam_button_callback_t callback);	12
exam_status_t exam_button_irq_start(exam_button_t button);	12
void exam_buttons_confirmation_ms(uint32_t milliseconds);	13
uint8_t exam_button_pressed(exam_button_t button);	13
uint32_t exam_buttons_pressed(void);	14
Joystick	14
exam_status_t exam_joystick_start(exam_joystick_callback_t callback);	14
uint32_t exam_joystick_read(void);	14
uint32_t exam_joystick_first(void);	15
void exam_joystick_reset_first(void);	15
Timers	15
exam_status_t exam_timer_every_ms(int timer, int milliseconds, exam_timer_callback_t callback);	15
exam_status_t exam_timer_every_hz(int timer, int hertz, exam_timer_callback_t callback);	16
exam_status_t exam_timer_clock_divider(uint8_t timer, uint8_t divider);	16
exam_status_t exam_timer_prescaler(uint8_t timer, uint32_t prescaler);	16
exam_status_t exam_timer_match(uint8_t timer, uint8_t match, uint32_t ticks, uint32_t actions);	17
exam_status_t exam_timer_start(uint8_t timer);	17

exam_status_t exam_timer_stop(uint8_t timer);	18
exam_status_t exam_timer_reset(uint8_t timer);	18
uint32_t exam_timer_count(uint8_t timer);	19
uint8_t exam_timer_match_happened(uint32_t flags, uint8_t match);	19
uint8_t exam_timer_capture_happened(uint32_t flags, uint8_t capture);	19
RIT	20
exam_status_t exam_rit_start(void);	20
void exam_rit_stop(void);	20
uint32_t exam_rit_ticks(void);	20
SysTick	21
exam_status_t exam_systick_every_ms(int milliseconds);	21
uint32_t exam_systick_ticks(void);	21
ADC and potentiometer	21
exam_status_t exam_pot_start(void);	21
exam_status_t exam_pot_read(int * value);	22
exam_status_t exam_adc_read(uint8_t channel, uint16_t * value);	22
DAC and analog output	23
exam_status_t exam_dac_write(int value);	23
exam_status_t exam_dac_percent(int percent);	23
void exam_dac_silence(void);	23
Interrupt communication	24
void exam_events_set(uint32_t bits);	24
uint32_t exam_events_take(uint32_t mask);	24
uint32_t exam_critical_enter(void);	24
void exam_critical_exit(uint32_t saved_primask);	25
Faults, SVC and hooks	25
uint32_t exam_self_test(void);	25

Precise driver and utility functions

26

Timers	26
board_status_t timer_every_ms(int timer, int milliseconds, timer_callback_t callback);	26
board_status_t timer_every_hz(int timer, int hertz, timer_callback_t callback);	26
SysTick	26
board_status_t systick_every_ms(int milliseconds);	26
ADC and potentiometer	27
board_status_t potentiometer_start(void);	27
board_status_t potentiometer_read(int * value);	27
DAC and analog output	28
board_status_t speaker_write(int value);	28
board_status_t speaker_write_percent(int percent);	28
External buttons	28
int int0_pressed(void);	28
int key1_pressed(void);	29
int key2_pressed(void);	29
Joystick	29
int joystick_up_pressed(void);	29

int joystick_down_pressed(void);	30
int joystick_left_pressed(void);	30
int joystick_right_pressed(void);	30
int joystick_button_pressed(void);	30
Program control	31
void board_init(void);	31
void board_idle(void);	31
LED output	31
board_status_t led_write_number(uint8_t led_number, uint8_t on);	31
board_status_t led_on(uint8_t led_number);	32
board_status_t led_off(uint8_t led_number);	32
void led_all_off(void);	32
board_status_t led_write_mask(uint8_t mask);	32
board_status_t led_toggle(uint8_t led_number);	33
uint8_t led_read_mask(void);	33
void led_write8(uint8_t value);	33
External buttons	34
void buttons_init(button_callback_t callback);	34
board_status_t button_irq_start(board_button_t button);	34
void buttons_set_confirmation_ms(uint32_t milliseconds);	34
uint32_t buttons_pressed_mask(void);	35
uint8_t button_is_pressed(board_button_t button);	35
Joystick	35
void joystick_init(joystick_callback_t callback);	35
uint32_t joystick_read(void);	36
uint8_t joystick_is_pressed(uint32_t direction);	36
uint32_t joystick_first_movement(void);	36
void joystick_reset_first_movement(void);	37
Timers	37
board_status_t timer_set_clock_divider(uint8_t timer, uint8_t divider);	37
uint32_t timer_peripheral_clock_hz(uint8_t timer);	37
board_status_t timer_set_prescaler(uint8_t timer, uint32_t prescaler);	38
uint32_t timer_counter_clock_hz(uint8_t timer);	38
board_status_t timer_calculate_match_for_frequency(uint8_t timer, uint32_t frequency_hz, uint32_t events_per_cycle, uint32_t * match_value);	38
board_status_t timer_configure_frequency(uint8_t timer, uint8_t match, uint32_t frequency_hz, uint32_t events_per_cycle, uint32_t actions);	39
board_status_t timer_start_periodic_interrupt_hz(uint8_t timer, uint32_t frequency_hz, uint32_t events_per_cycle, timer_callback_t callback);	40
board_status_t timer_start_periodic_interrupt_ms(uint8_t timer, uint32_t period_ms, timer_callback_t callback);	40
uint8_t timer_match_occurred(uint32_t pending_flags, uint8_t match);	41
uint8_t timer_capture_occurred(uint32_t pending_flags, uint8_t capture);	41
board_status_t timer_configure_match(uint8_t timer, uint8_t match, uint32_t value, uint32_t actions);	41
board_status_t timer_configure_capture(uint8_t timer, uint8_t capture, timer_capture_edge_t edge, uint8_t interrupt_enable);	42
board_status_t timer_read_capture(uint8_t timer, uint8_t capture, uint32_t * value);	42
board_status_t timer_set_counter_mode(uint8_t timer, timer_counter_mode_t mode, uint8_t capture_input);	43
board_status_t timer_configure_external_match(uint8_t timer, uint8_t match, timer_external_match_t action, uint8_t initial_state);	43
board_status_t timer_start(uint8_t timer);	44
board_status_t timer_stop(uint8_t timer);	44
board_status_t timer_reset(uint8_t timer);	45

board_status_t timer_set_callback(uint8_t timer, timer_callback_t callback);	45
uint32_t timer_read_counter(uint8_t timer);	45
RIT	46
board_status_t rit_scheduler_start(void);	46
void rit_scheduler_stop(void);	46
uint32_t rit_scheduler_ticks(void);	46
board_status_t rit_raw_configure(uint32_t compare, uint32_t mask, uint8_t clear_on_match);	47
board_status_t rit_raw_start(void);	47
void rit_raw_stop(void);	48
uint32_t rit_raw_read(void);	48
SysTick	48
board_status_t systick_configure(uint32_t reload, uint8_t periodic, uint8_t interrupt_enable);	48
board_status_t systick_start_periodic_ms(uint32_t period_ms);	49
board_status_t systick_set_clock_source(uint8_t processor_clock);	49
uint32_t systick_calibration_value(void);	49
uint8_t systick_has_precise_calibration(void);	50
uint32_t systick_ticks(void);	50
ADC and potentiometer	50
board_status_t adc_init(uint8_t channel, uint32_t peripheral_clock_hz);	50
board_status_t adc_configure_trigger(adc_trigger_t trigger, uint8_t falling_edge);	51
board_status_t adc_start_conversion(void);	51
board_status_t adc_start_burst(uint32_t channel_mask);	51
void adc_stop_burst(void);	52
board_status_t adc_read_channel(uint8_t channel, adc_sample_t * sample);	52
board_status_t adc_read_value(uint8_t channel, uint16_t * value);	52
uint32_t adc_clock_hz(void);	53
DAC and analog output	53
board_status_t dac_init(void);	53
board_status_t dac_write(uint16_t value);	54
void dac_silence(void);	54
board_status_t dac_validate_update_rate(uint32_t update_hz);	54
board_status_t dac_play_samples(const uint16_t * samples, uint32_t count, uint32_t update_hz, uint8_t timer);	55
Interrupt communication	55
uint32_t critical_enter(void);	55
void critical_exit(uint32_t previous_primask);	55
void event_flags_set(uint32_t flags);	56
uint32_t event_flags_take(uint32_t mask);	56
Faults, SVC and hooks	56
void fault_traps_configure(void);	56
void svc_dispatch(uint8_t service_number, svc_context_t * context);	57
Convenience and control	57
void * board_direct_register(const char * name);	57
Faults, SVC and hooks	57
void exam_user_10ms_hook(void);	58
uint32_t board_self_test_run(void);	58

LED output	59
void LED_init(void);	59
void LED_deinit(void);	59
void LED_On(unsigned int bit_number);	59
void LED_Off(unsigned int bit_number);	59
void LED_Out(unsigned int value);	60
External buttons	60
void BUTTON_init(void);	60
Timers	60
uint32_t init_timer(uint8_t timer_num, uint32_t interval);	60
void enable_timer(uint8_t timer_num);	61
void disable_timer(uint8_t timer_num);	61
void reset_timer(uint8_t timer_num);	62
RIT	62
uint32_t init_RIT(uint32_t interval);	62
void enable_RIT(void);	62
void disable_RIT(void);	63
void reset_RIT(void);	63
ADC and potentiometer	63
void ADC_init(void);	63
void ADC_start_conversion(void);	64

Exact handlers and Reset_Handler **65**

Exact timer handler	65
Exact external-button handler	65
Exact ADC handler	65
Reset_Handler	65

Alphabetical function index **66**

How to use this guide

Start with the recommended exam_* layer when a peripheral is only supporting the algorithm. Use the precise layer when MR, MCR, capture, counter mode, RIT compare/mask, ADC trigger or another hardware detail is graded. Use direct registers and an exact handler only when the paper explicitly asks for them.

All choices use the same CA Exam target. You never submit multiple answer styles.

Status values

Exam	Precise	Meaning
EXAM_OK	BOARD_OK	Request succeeded
EXAM_INVALID	BOARD_INVALID	Null pointer or invalid combination
EXAM_BUSY	BOARD_BUSY	Exact vector, resource or mode conflict
EXAM_RANGE	BOARD_RANGE	Number, rate, period, channel or value is outside its range
EXAM_NOT_READY	BOARD_NOT_READY	No fresh cached result exists yet
-	BOARD_NOT_ENABLED	Reserved precise-layer result; no excluded optional peripheral is present in this kit

Interrupt ownership

A vector can have only one linked handler. The template owns common handlers by default and calls your callback. If the paper explicitly requires an exact handler, enable only its EXAM_OWN_*_HANDLER switch. Callback helpers for that same vector return BUSY; other vectors and other timers keep working.

Important types and constants

uint8_t, uint16_t and uint32_t are unsigned 8-, 16- and 32-bit values. A parameter containing * is an address: create the variable and pass it with &, for example exam_pot_read(&value;).

External buttons are EXAM_BUTTON_INT0, EXAM_BUTTON_KEY1 and EXAM_BUTTON_KEY2. Joystick results are ORed masks. Timer actions may be ORed from EXAM_TIMER_INTERRUPT, EXAM_TIMER_RESET and EXAM_TIMER_STOP.

```
void exam_button_event(exam_button_t button, exam_button_event_t event);
void exam_joystick_event(uint32_t current, uint32_t changed);
void exam_timer_event(uint8_t timer, uint32_t flags);
```

Functions you write

Answer structure and callbacks

void exam_user_init(void);

```
void exam_user_init(void);
```

Purpose: The one-time answer setup function called by the internal main after exam_init.

Parameters: None.

Returns: No return value.

Setup and ownership: This function is already declared and has a starter body in exam_user.c; edit that body instead of creating a second definition.

Hardware/shared side effects: Changes only the answer state and peripherals used inside the body. Callback forms run inside interrupt context.

Copyable example:

```
void exam_user_init(void)
{
    exam_timer_every_ms(0, 1000, exam_timer_event);
}
```

Common mistake: Do not write main again. Keep callback bodies short and move long work to exam_user_loop.

void exam_user_loop(void);

```
void exam_user_loop(void);
```

Purpose: The repeating foreground function called forever by the internal main.

Parameters: None.

Returns: No return value.

Setup and ownership: This function is already declared and has a starter body in exam_user.c; edit that body instead of creating a second definition.

Hardware/shared side effects: Changes only the answer state and peripherals used inside the body. Callback forms run inside interrupt context.

Copyable example:

```
void exam_user_loop(void)
{
    if (exam_events_take(EXAM_EVENT_TIMER)) { /* longer work */ }
}
```

Common mistake: Do not write main again. Keep callback bodies short and move long work to exam_user_loop.

void exam_button_event(exam_button_t button, exam_button_event_t event);

```
void exam_button_event(exam_button_t button, exam_button_event_t event);
```

Purpose: The supplied callback shape for confirmed external-button press and release events.

Parameters:

Declaration	Meaning
exam_button_t button	One of INT0, KEY1 or KEY2 enum values.
exam_button_event_t event	Confirmed press or release event.

Returns: No return value.

Setup and ownership: This function is already declared and has a starter body in exam_user.c; edit that body instead of creating a second definition.

Hardware/shared side effects: Changes only the answer state and peripherals used inside the body. Callback forms run inside interrupt context.

Copyable example:

```
void exam_button_event(exam_button_t b, exam_button_event_t e)
{
    if (b == EXAM_BUTTON_INT0 && e == EXAM_PRESS) exam_events_set(EXAM_EVENT_BUTTON);
}
```

Common mistake: Do not write main again. Keep callback bodies short and move long work to exam_user_loop.

void exam_joystick_event(uint32_t current, uint32_t changed);

```
void exam_joystick_event(uint32_t current, uint32_t changed);
```

Purpose: The supplied callback shape for joystick-state changes.

Parameters:

Declaration	Meaning
uint32_t current	Current joystick direction mask.
uint32_t changed	Mask of joystick bits that changed since the last service.

Returns: No return value.

Setup and ownership: This function is already declared and has a starter body in exam_user.c; edit that body instead of creating a second definition.

Hardware/shared side effects: Changes only the answer state and peripherals used inside the body. Callback forms run inside interrupt context.

Copyable example:

```
void exam_joystick_event(uint32_t current, uint32_t changed)
{
    (void)changed;
    if (current & EXAM_JOY_UP) exam_events_set(EXAM_EVENT_JOYSTICK);
}
```

Common mistake: Do not write main again. Keep callback bodies short and move long work to exam_user_loop.

void exam_timer_event(uint8_t timer, uint32_t flags);

```
void exam_timer_event(uint8_t timer, uint32_t flags);
```

Purpose: The supplied callback shape shared by Timer 0 through Timer 3.

Parameters:

Declaration	Meaning
uint8_t timer	Hardware timer number 0 through 3.
uint32_t flags	Bit mask supplied or consumed by this function.

Returns: No return value.

Setup and ownership: This function is already declared and has a starter body in exam_user.c; edit that body instead of creating a second definition.

Hardware/shared side effects: Changes only the answer state and peripherals used inside the body. Callback forms run inside interrupt context.

Copyable example:

```
void exam_timer_event(uint8_t timer, uint32_t flags)
{
    if (timer == 0 && exam_timer_match_happened(flags, 0)) exam_events_set(EXAM_EVENT_TIMER);
}
```

Common mistake: Do not write main again. Keep callback bodies short and move long work to exam_user_loop.

Recommended exam_* functions

Program control

void exam_init(void);

```
void exam_init(void);
```

Purpose: Initialize the system clock, LED GPIO and configurable fault support. The internal main calls it once before the answer code. It is the beginner-facing wrapper for `board_init()`.

Parameters: None.

Returns: No return value. Observe the documented hardware or shared-state effect.

Setup and ownership: The internal main already calls this. Do not call it again in a normal answer.

Hardware/shared side effects: May change system clock setup, LED GPIO, SCB fault controls, or CPU sleep state as described.

Copyable example:

```
/* Called internally before exam_user_init(). */
```

Common mistake: Do not call it twice or assume that it initialized buttons, timers, RIT, joystick, ADC or DAC.

void exam_idle(void);

```
void exam_idle(void);
```

Purpose: Run the end of one foreground-loop iteration. It is a NOP by default and optionally executes WFI when explicitly configured. It is the beginner-facing wrapper for `board_idle()`.

Parameters: None.

Returns: No return value. Observe the documented hardware or shared-state effect.

Setup and ownership: The internal main already calls this. Do not call it again in a normal answer.

Hardware/shared side effects: May change system clock setup, LED GPIO, SCB fault controls, or CPU sleep state as described.

Copyable example:

```
/* Called internally after each exam_user_loop(). */
```

Common mistake: Enable WFI only when an enabled interrupt can wake the processor; the default NOP is safer for polling.

LED output

exam_status_t exam_led_on(uint8_t printed_number);

```
exam_status_t exam_led_on(uint8_t printed_number);
```

Purpose: Turn on one LED selected by its printed number 4 through 11. It is the beginner-facing wrapper for `led_on()`.

Parameters:

Declaration	Meaning
uint8_t printed_number	Printed LandTiger LED number 4 through 11.

Returns: EXAM_OK on success. Depending on the call, EXAM_RANGE, EXAM_INVALID, EXAM_BUSY or EXAM_NOT_READY reports a problem without guessing.

Setup and ownership: No extra setup is required after the internal exam initialization. LED APIs use P2.0 through P2.7.

Hardware/shared side effects: Writes GPIO2 direction/set/clear state and updates the stored LED value.

Copyable example:

```
exam_led_on(4);
```

Common mistake: Do not confuse printed LED numbers 4..11 with legacy GPIO bit numbers 0..7.

exam_status_t exam_led_off(uint8_t printed_number);

```
exam_status_t exam_led_off(uint8_t printed_number);
```

Purpose: Turn off one LED selected by its printed number 4 through 11. It is the beginner-facing wrapper for `led_off()`.

Parameters:

Declaration	Meaning
uint8_t printed_number	Printed LandTiger LED number 4 through 11.

Returns: EXAM_OK on success. Depending on the call, EXAM_RANGE, EXAM_INVALID, EXAM_BUSY or EXAM_NOT_READY reports a problem without guessing.

Setup and ownership: No extra setup is required after the internal exam initialization. LED APIs use P2.0 through P2.7.

Hardware/shared side effects: Writes GPIO2 direction/set/clear state and updates the stored LED value.

Copyable example:

```
exam_led_off(4);
```

Common mistake: Do not confuse printed LED numbers 4..11 with legacy GPIO bit numbers 0..7.

exam_status_t exam_led_toggle(uint8_t printed_number);

```
exam_status_t exam_led_toggle(uint8_t printed_number);
```

Purpose: Invert one LED selected by its printed number. It is the beginner-facing wrapper for `led_toggle()`.

Parameters:

Declaration	Meaning
uint8_t printed_number	Printed LandTiger LED number 4 through 11.

Returns: EXAM_OK on success. Depending on the call, EXAM_RANGE, EXAM_INVALID, EXAM_BUSY or EXAM_NOT_READY reports a problem without guessing.

Setup and ownership: No extra setup is required after the internal exam initialization. LED APIs use P2.0 through P2.7.

Hardware/shared side effects: Writes GPIO2 direction/set/clear state and updates the stored LED value.

Copyable example:

```
exam_led_toggle(4);
```

Common mistake: Do not confuse printed LED numbers 4..11 with legacy GPIO bit numbers 0..7.

exam_status_t exam_led_write(uint8_t mask);

```
exam_status_t exam_led_write(uint8_t mask);
```

Purpose: Write the complete eight-bit LED row; bit 0 maps to P2.0 and printed LED4. It is the beginner-facing wrapper for `led_write_mask()`.

Parameters:

Declaration	Meaning
uint8_t mask	Bit mask interpreted by this function.

Returns: EXAM_OK on success. Depending on the call, EXAM_RANGE, EXAM_INVALID, EXAM_BUSY or EXAM_NOT_READY reports a problem without guessing.

Setup and ownership: No extra setup is required after the internal exam initialization. LED APIs use P2.0 through P2.7.

Hardware/shared side effects: Writes GPIO2 direction/set/clear state and updates the stored LED value.

Copyable example:

```
exam_led_write(0xA5);
```

Common mistake: Do not confuse printed LED numbers 4..11 with legacy GPIO bit numbers 0..7.

uint8_t exam_led_read(void);

```
uint8_t exam_led_read(void);
```

Purpose: Read the current eight LED output bits. It is the beginner-facing wrapper for `led_read_mask()`.

Parameters: None.

Returns: Returns the current low eight GPIO2 output bits.

Setup and ownership: No extra setup is required after the internal exam initialization. LED APIs use P2.0 through P2.7.

Hardware/shared side effects: Writes GPIO2 direction/set/clear state and updates the stored LED value.

Copyable example:

```
uint8_t shown = exam_led_read();
```

Common mistake: Do not confuse printed LED numbers 4..11 with legacy GPIO bit numbers 0..7.

void exam_leds_off(void);

```
void exam_leds_off(void);
```

Purpose: Turn off all eight LEDs in one operation. It is the beginner-facing wrapper for `led_all_off()`.

Parameters: None.

Returns: No return value. Observe the documented hardware or shared-state effect.

Setup and ownership: No extra setup is required after the internal exam initialization. LED APIs use P2.0 through P2.7.

Hardware/shared side effects: Writes GPIO2 direction/set/clear state and updates the stored LED value.

Copyable example:

```
exam_leds_off();
```

Common mistake: Do not confuse printed LED numbers 4..11 with legacy GPIO bit numbers 0..7.

External buttons

exam_status_t exam_buttons_start(exam_button_callback_t callback);

```
exam_status_t exam_buttons_start(exam_button_callback_t callback);
```

Purpose: Configure INT0, KEY1 and KEY2 as falling-edge external interrupts and register an optional confirmed-event callback. This exam wrapper also starts the normal RIT scheduler and reports a mode conflict. It is the beginner-facing wrapper for `buttons_init()`.

Parameters:

Declaration	Meaning
<code>exam_button_callback_t callback</code>	Function called inside the matching interrupt; keep it short.

Returns: EXAM_OK on success. Depending on the call, EXAM_RANGE, EXAM_INVALID, EXAM_BUSY or EXAM_NOT_READY reports a problem without guessing.

Setup and ownership: Call once in `exam_user_init`. This wrapper starts the normal 10 ms RIT service before configuring the buttons.

Hardware/shared side effects: May change PINSEL4, GPIO2 direction, EXTMODE, EXTPOLAR, EXTINT and the matching NVIC enable state.

Copyable example:

```
if (exam_buttons_start(exam_button_event) != EXAM_OK) { /* configuration conflict */ }
```

Common mistake: Do not add a blocking debounce delay. Callback confirmation needs the normal RIT scheduler; polling does not.

exam_status_t exam_button_irq_start(exam_button_t button);

```
exam_status_t exam_button_irq_start(exam_button_t button);
```

Purpose: Configure and enable one exact EINT vector without adding debounce, callbacks or RIT scheduling. It is the beginner-facing wrapper for `button_irq_start()`.

Parameters:

Declaration	Meaning
<code>exam_button_t button</code>	One of INT0, KEY1 or KEY2 enum values.

Returns: EXAM_OK on success. Depending on the call, EXAM_RANGE, EXAM_INVALID, EXAM_BUSY or EXAM_NOT_READY reports a problem without guessing.

Setup and ownership: Enable the matching EXAM_OWN_EINTx_HANDLER switch, call this once, then provide that exact EINTx handler and clear EXTINT.

Hardware/shared side effects: May change PINSEL4, GPIO2 direction, EXTMODE, EXTPOLAR, EXTINT and the matching NVIC enable state.

Copyable example:

```
exam_button_irq_start(EXAM_BUTTON_INT0);
```

Common mistake: Do not enable the exact handler without its ownership switch, and clear the matching EXTINT W1C bit in the handler.

void exam_buttons_confirmation_ms(uint32_t milliseconds);

```
void exam_buttons_confirmation_ms(uint32_t milliseconds);
```

Purpose: Change the software confirmation interval used by the external-button service. It is the beginner-facing wrapper for `buttons_set_confirmation_ms()`.

Parameters:

Declaration	Meaning
<code>uint32_t milliseconds</code>	Confirmation interval. Use at least 10 and a multiple of 10 ms.

Returns: No return value. Observe the documented hardware or shared-state effect.

Setup and ownership: Polling needs no interrupt. Inputs are active-low.

Hardware/shared side effects: May change PINSEL4, GPIO2 direction, EXTMODE, EXTPOLAR, EXTINT and the matching NVIC enable state.

Copyable example:

```
exam_buttons_confirmation_ms(50);
```

Common mistake: Do not add a blocking debounce delay. Callback confirmation needs the normal RIT scheduler; polling does not.

uint8_t exam_button_pressed(exam_button_t button);

```
uint8_t exam_button_pressed(exam_button_t button);
```

Purpose: Poll one selected external button and return 1 while it is held. It is the beginner-facing wrapper for `button_is_pressed()`.

Parameters:

Declaration	Meaning
<code>exam_button_t button</code>	One of INT0, KEY1 or KEY2 enum values.

Returns: Returns 1 when the tested condition is true and 0 otherwise.

Setup and ownership: Polling needs no interrupt. Inputs are active-low.

Hardware/shared side effects: May change PINSEL4, GPIO2 direction, EXTMODE, EXTPOLAR, EXTINT and the matching NVIC enable state.

Copyable example:

```
if (exam_button_pressed(EXAM_BUTTON_INT0)) { /* held */ }
```

Common mistake: Do not add a blocking debounce delay. Callback confirmation needs the normal RIT scheduler; polling does not.

uint32_t exam_buttons_pressed(void);

```
uint32_t exam_buttons_pressed(void);
```

Purpose: Poll all three external buttons and return a bit mask of the buttons currently held. It is the beginner-facing wrapper for `buttons_pressed_mask()`.

Parameters: None.

Returns: Returns bits 0, 1 and 2 for INT0, KEY1 and KEY2 currently held.

Setup and ownership: Polling needs no interrupt. Inputs are active-low.

Hardware/shared side effects: May change PINSEL4, GPIO2 direction, EXTMODE, EXTPOLAR, EXTINT and the matching NVIC enable state.

Copyable example:

```
uint32_t held = exam_buttons_pressed();
```

Common mistake: Do not add a blocking debounce delay. Callback confirmation needs the normal RIT scheduler; polling does not.

Joystick

exam_status_t exam_joystick_start(exam_joystick_callback_t callback);

```
exam_status_t exam_joystick_start(exam_joystick_callback_t callback);
```

Purpose: Configure the joystick inputs and register an optional change callback. This exam wrapper also starts the normal RIT scheduler and reports a mode conflict. It is the beginner-facing wrapper for `joystick_init()`.

Parameters:

Declaration	Meaning
<code>exam_joystick_callback_t callback</code>	Function called inside the matching interrupt; keep it short.

Returns: EXAM_OK on success. Depending on the call, EXAM_RANGE, EXAM_INVALID, EXAM_BUSY or EXAM_NOT_READY reports a problem without guessing.

Setup and ownership: Call once in `exam_user_init`. This wrapper starts the normal 10 ms RIT service.

Hardware/shared side effects: Reads or configures GPIO1 and may update callback and first-movement state.

Copyable example:

```
exam_joystick_start(exam_joystick_event);
```

Common mistake: Direction values are masks. Test with `&`, not equality, because combinations can occur.

uint32_t exam_joystick_read(void);

```
uint32_t exam_joystick_read(void);
```

Purpose: Return the current five-way joystick state as a mask, preserving combinations. It is the beginner-facing wrapper for `joystick_read()`.

Parameters: None.

Returns: Returns an ORed mask of all currently active joystick directions.

Setup and ownership: Polling reads GPIO directly. First-movement memory updates from the normal RIT service.

Hardware/shared side effects: Reads or configures GPIO1 and may update callback and first-movement state.

Copyable example:

```
uint32_t directions = exam_joystick_read();
```

Common mistake: Direction values are masks. Test with `&`, not equality, because combinations can occur.

uint32_t exam_joystick_first(void);

```
uint32_t exam_joystick_first(void);
```

Purpose: Return the first nonzero joystick movement remembered after the last reset. It is the beginner-facing wrapper for `joystick_first_movement()`.

Parameters: None.

Returns: Returns the remembered first direction mask, or 0 if none has been serviced.

Setup and ownership: Polling reads GPIO directly. First-movement memory updates from the normal RIT service.

Hardware/shared side effects: Reads or configures GPIO1 and may update callback and first-movement state.

Copyable example:

```
uint32_t first = exam_joystick_first();
```

Common mistake: Direction values are masks. Test with `&`, not equality, because combinations can occur.

void exam_joystick_reset_first(void);

```
void exam_joystick_reset_first(void);
```

Purpose: Clear first-movement memory before starting a new trial or round. It is the beginner-facing wrapper for `joystick_reset_first_movement()`.

Parameters: None.

Returns: No return value. Observe the documented hardware or shared-state effect.

Setup and ownership: Polling reads GPIO directly. First-movement memory updates from the normal RIT service.

Hardware/shared side effects: Reads or configures GPIO1 and may update callback and first-movement state.

Copyable example:

```
exam_joystick_reset_first();
```

Common mistake: Direction values are masks. Test with `&`, not equality, because combinations can occur.

Timers

exam_status_t exam_timer_every_ms(int timer, int milliseconds, exam_timer_callback_t callback);

```
exam_status_t exam_timer_every_ms(
    int timer,
    int milliseconds,
    exam_timer_callback_t callback
);
```

Purpose: Configure MR0 as a repeating interrupt every requested number of milliseconds, attach a callback, reset the counter on each match, and start the selected timer. It is the beginner-facing wrapper for `timer_every_ms()`.

Parameters:

Declaration	Meaning
int timer	Hardware timer number 0 through 3.
int milliseconds	Positive time in milliseconds.
exam_timer_callback_t callback	Function called inside the matching interrupt; keep it short.

Returns: EXAM_OK on success. Depending on the call, EXAM_RANGE, EXAM_INVALID, EXAM_BUSY or EXAM_NOT_READY reports a problem without guessing.

Setup and ownership: The selected timer must use the template's handler. Owning that `TIMERn_IRQHandler` makes callback registration return BUSY. Other timers remain independent.

Hardware/shared side effects: May power the selected timer and change PCLKSEL, TCR, TC, PC, PR, CTCR, MCR, CCR, EMR, MR, IR and NVIC state.

Copyable example:

```
exam_timer_every_ms(0, 1000, exam_timer_event);
```

Common mistake: Check PCLK, PR+1, units, timer number and W1C IR clearing. Never combine a callback helper with exact ownership of the same TIMERN vector.

exam_status_t exam_timer_every_hz(int timer, int hertz, exam_timer_callback_t callback);

```
exam_status_t exam_timer_every_hz(int timer, int hertz, exam_timer_callback_t callback);
```

Purpose: Configure MR0 for one callback per requested cycle frequency and start the selected timer. It is the beginner-facing wrapper for `timer_every_hz()`.

Parameters:

Declaration	Meaning
int timer	Hardware timer number 0 through 3.
int hertz	Positive frequency in hertz.
exam_timer_callback_t callback	Function called inside the matching interrupt; keep it short.

Returns: EXAM_OK on success. Depending on the call, EXAM_RANGE, EXAM_INVALID, EXAM_BUSY or EXAM_NOT_READY reports a problem without guessing.

Setup and ownership: The selected timer must use the template's handler. Owning that TIMERN_IRQHandler makes callback registration return BUSY. Other timers remain independent.

Hardware/shared side effects: May power the selected timer and change PCLKSEL, TCR, TC, PC, PR, CTCR, MCR, CCR, EMR, MR, IR and NVIC state.

Copyable example:

```
exam_timer_every_hz(1, 100, exam_timer_event);
```

Common mistake: Check PCLK, PR+1, units, timer number and W1C IR clearing. Never combine a callback helper with exact ownership of the same TIMERN vector.

exam_status_t exam_timer_clock_divider(uint8_t timer, uint8_t divider);

```
exam_status_t exam_timer_clock_divider(uint8_t timer, uint8_t divider);
```

Purpose: Select the peripheral-clock divider for Timer 0, 1, 2 or 3 while that timer is stopped. It is the beginner-facing wrapper for `timer_set_clock_divider()`.

Parameters:

Declaration	Meaning
uint8_t timer	Hardware timer number 0 through 3.
uint8_t divider	Peripheral-clock divider 1, 2, 4 or 8.

Returns: EXAM_OK on success. Depending on the call, EXAM_RANGE, EXAM_INVALID, EXAM_BUSY or EXAM_NOT_READY reports a problem without guessing.

Setup and ownership: Use timer 0 through 3. Configure its clock, prescaler and mode before starting when the question grades those values.

Hardware/shared side effects: May power the selected timer and change PCLKSEL, TCR, TC, PC, PR, CTCR, MCR, CCR, EMR, MR, IR and NVIC state.

Copyable example:

```
exam_timer_clock_divider(0, 4);
```

Common mistake: Check PCLK, PR+1, units, timer number and W1C IR clearing. Never combine a callback helper with exact ownership of the same TIMERN vector.

exam_status_t exam_timer_prescaler(uint8_t timer, uint32_t prescaler);

```
exam_status_t exam_timer_prescaler(uint8_t timer, uint32_t prescaler);
```

Purpose: Write PR so TC increments once every PR+1 peripheral-clock ticks. It is the beginner-facing wrapper for `timer_set_prescaler()`.

Parameters:

Declaration	Meaning
uint8_t timer	Hardware timer number 0 through 3.
uint32_t prescaler	PR value. TC divides its input clock by prescaler+1.

Returns: EXAM_OK on success. Depending on the call, EXAM_RANGE, EXAM_INVALID, EXAM_BUSY or EXAM_NOT_READY reports a problem without guessing.

Setup and ownership: Use timer 0 through 3. Configure its clock, prescaler and mode before starting when the question grades those values.

Hardware/shared side effects: May power the selected timer and change PCLKSEL, TCR, TC, PC, PR, CTCR, MCR, CCR, EMR, MR, IR and NVIC state.

Copyable example:

```
exam_timer_prescaler(0, 0);
```

Common mistake: Check PCLK, PR+1, units, timer number and W1C IR clearing. Never combine a callback helper with exact ownership of the same TIMERN vector.

exam_status_t exam_timer_match(uint8_t timer, uint8_t match, uint32_t ticks, uint32_t actions);

```
exam_status_t exam_timer_match(
    uint8_t timer,
    uint8_t match,
    uint32_t ticks,
    uint32_t actions
);
```

Purpose: Write MR0 through MR3 and program the matching MCR interrupt, reset and stop actions. It is the beginner-facing wrapper for timer_configure_match().

Parameters:

Declaration	Meaning
uint8_t timer	Hardware timer number 0 through 3.
uint8_t match	Match channel 0 through 3.
uint32_t ticks	Exact timer match count in TC ticks.
uint32_t actions	ORed interrupt, reset and stop action bits.

Returns: EXAM_OK on success. Depending on the call, EXAM_RANGE, EXAM_INVALID, EXAM_BUSY or EXAM_NOT_READY reports a problem without guessing.

Setup and ownership: Use timer 0 through 3. Configure its clock, prescaler and mode before starting when the question grades those values.

Hardware/shared side effects: May power the selected timer and change PCLKSEL, TCR, TC, PC, PR, CTCR, MCR, CCR, EMR, MR, IR and NVIC state.

Copyable example:

```
exam_timer_match(0, 0, 25000000, EXAM_TIMER_INTERRUPT | EXAM_TIMER_RESET);
```

Common mistake: Check PCLK, PR+1, units, timer number and W1C IR clearing. Never combine a callback helper with exact ownership of the same TIMERN vector.

exam_status_t exam_timer_start(uint8_t timer);

```
exam_status_t exam_timer_start(uint8_t timer);
```

Purpose: Start or resume one hardware timer by setting TCR enable. It is the beginner-facing wrapper for timer_start().

Parameters:

Declaration	Meaning
uint8_t timer	Hardware timer number 0 through 3.

Returns: EXAM_OK on success. Depending on the call, EXAM_RANGE, EXAM_INVALID, EXAM_BUSY or EXAM_NOT_READY reports a problem without guessing.

Setup and ownership: Use timer 0 through 3. Configure its clock, prescaler and mode before starting when the question grades those values.

Hardware/shared side effects: May power the selected timer and change PCLKSEL, TCR, TC, PC, PR, CTCR, MCR, CCR, EMR, MR, IR and NVIC state.

Copyable example:

```
exam_timer_start(0);
```

Common mistake: Check PCLK, PR+1, units, timer number and W1C IR clearing. Never combine a callback helper with exact ownership of the same TIMERN vector.

exam_status_t exam_timer_stop(uint8_t timer);

```
exam_status_t exam_timer_stop(uint8_t timer);
```

Purpose: Stop one hardware timer without clearing its current TC value. It is the beginner-facing wrapper for timer_stop().

Parameters:

Declaration	Meaning
uint8_t timer	Hardware timer number 0 through 3.

Returns: EXAM_OK on success. Depending on the call, EXAM_RANGE, EXAM_INVALID, EXAM_BUSY or EXAM_NOT_READY reports a problem without guessing.

Setup and ownership: Use timer 0 through 3. Configure its clock, prescaler and mode before starting when the question grades those values.

Hardware/shared side effects: May power the selected timer and change PCLKSEL, TCR, TC, PC, PR, CTCR, MCR, CCR, EMR, MR, IR and NVIC state.

Copyable example:

```
exam_timer_stop(0);
```

Common mistake: Check PCLK, PR+1, units, timer number and W1C IR clearing. Never combine a callback helper with exact ownership of the same TIMERN vector.

exam_status_t exam_timer_reset(uint8_t timer);

```
exam_status_t exam_timer_reset(uint8_t timer);
```

Purpose: Pulse the timer reset bit, clear TC and PC, and leave the timer stopped. It is the beginner-facing wrapper for timer_reset().

Parameters:

Declaration	Meaning
uint8_t timer	Hardware timer number 0 through 3.

Returns: EXAM_OK on success. Depending on the call, EXAM_RANGE, EXAM_INVALID, EXAM_BUSY or EXAM_NOT_READY reports a problem without guessing.

Setup and ownership: Use timer 0 through 3. Configure its clock, prescaler and mode before starting when the question grades those values.

Hardware/shared side effects: May power the selected timer and change PCLKSEL, TCR, TC, PC, PR, CTCR, MCR, CCR, EMR, MR, IR and NVIC state.

Copyable example:

```
exam_timer_reset(0);
```

Common mistake: Check PCLK, PR+1, units, timer number and W1C IR clearing. Never combine a callback helper with exact ownership of the same TIMERN vector.

uint32_t exam_timer_count(uint8_t timer);

```
uint32_t exam_timer_count(uint8_t timer);
```

Purpose: Read the current TC value of a timer, including a free-running timer. It is the beginner-facing wrapper for `timer_read_counter()`.

Parameters:

Declaration	Meaning
uint8_t timer	Hardware timer number 0 through 3.

Returns: Returns TC, or 0 for an invalid timer.

Setup and ownership: Use timer 0 through 3. Configure its clock, prescaler and mode before starting when the question grades those values.

Hardware/shared side effects: May power the selected timer and change PCLKSEL, TCR, TC, PC, PR, CTCR, MCR, CCR, EMR, MR, IR and NVIC state.

Copyable example:

```
uint32_t elapsed = exam_timer_count(1);
```

Common mistake: Check PCLK, PR+1, units, timer number and W1C IR clearing. Never combine a callback helper with exact ownership of the same TIMERN vector.

uint8_t exam_timer_match_happened(uint32_t flags, uint8_t match);

```
uint8_t exam_timer_match_happened(uint32_t flags, uint8_t match);
```

Purpose: Test callback pending flags for MR0, MR1, MR2 or MR3. It is the beginner-facing wrapper for `timer_match_occurred()`.

Parameters:

Declaration	Meaning
uint32_t flags	Bit mask supplied or consumed by this function.
uint8_t match	Match channel 0 through 3.

Returns: Returns 1 when the tested condition is true and 0 otherwise.

Setup and ownership: Use timer 0 through 3. Configure its clock, prescaler and mode before starting when the question grades those values.

Hardware/shared side effects: May power the selected timer and change PCLKSEL, TCR, TC, PC, PR, CTCR, MCR, CCR, EMR, MR, IR and NVIC state.

Copyable example:

```
if (exam_timer_match_happened(flags, 0)) { /* MR0 */ }
```

Common mistake: Check PCLK, PR+1, units, timer number and W1C IR clearing. Never combine a callback helper with exact ownership of the same TIMERN vector.

uint8_t exam_timer_capture_happened(uint32_t flags, uint8_t capture);

```
uint8_t exam_timer_capture_happened(uint32_t flags, uint8_t capture);
```

Purpose: Test callback pending flags for CR0 or CR1. It is the beginner-facing wrapper for `timer_capture_occurred()`.

Parameters:

Declaration	Meaning
uint32_t flags	Bit mask supplied or consumed by this function.
uint8_t capture	Capture channel 0 or 1.

Returns: Returns 1 when the tested condition is true and 0 otherwise.

Setup and ownership: Use timer 0 through 3. Configure its clock, prescaler and mode before starting when the question grades those values.

Hardware/shared side effects: May power the selected timer and change PCLKSEL, TCR, TC, PC, PR, CTCR, MCR, CCR, EMR, MR, IR and NVIC state.

Copyable example:

```
if (exam_timer_capture_happened(flags, 1)) { /* CR1 */ }
```

Common mistake: Check PCLK, PR+1, units, timer number and W1C IR clearing. Never combine a callback helper with exact ownership of the same TIMERN vector.

RIT

exam_status_t exam_rit_start(void);

```
exam_status_t exam_rit_start(void);
```

Purpose: Configure and start the normal 10 ms RIT service used by confirmed buttons, joystick callbacks and the user hook. It is the beginner-facing wrapper for `rit_scheduler_start()`.

Parameters: None.

Returns: EXAM_OK on success. Depending on the call, EXAM_RANGE, EXAM_INVALID, EXAM_BUSY or EXAM_NOT_READY reports a problem without guessing.

Setup and ownership: Scheduler and direct RIT modes are compile-time exclusive. Exact RIT ownership also excludes the scheduler callback path.

Hardware/shared side effects: May power RIT and change PCLKSEL1, RICOMPVAL, RIMASK, RICOUNTER, RICTRL and NVIC state.

Copyable example:

```
exam_rit_start();
```

Common mistake: Do not mix scheduler mode, direct RIT mode and an exact RIT handler. Only one model can own the RIT vector.

void exam_rit_stop(void);

```
void exam_rit_stop(void);
```

Purpose: Stop the RIT counter used by the normal scheduler. It is the beginner-facing wrapper for `rit_scheduler_stop()`.

Parameters: None.

Returns: No return value. Observe the documented hardware or shared-state effect.

Setup and ownership: Scheduler and direct RIT modes are compile-time exclusive. Exact RIT ownership also excludes the scheduler callback path.

Hardware/shared side effects: May power RIT and change PCLKSEL1, RICOMPVAL, RIMASK, RICOUNTER, RICTRL and NVIC state.

Copyable example:

```
exam_rit_stop();
```

Common mistake: Do not mix scheduler mode, direct RIT mode and an exact RIT handler. Only one model can own the RIT vector.

uint32_t exam_rit_ticks(void);

```
uint32_t exam_rit_ticks(void);
```

Purpose: Return the number of 10 ms scheduler interrupts handled since reset. It is the beginner-facing wrapper for `rit_scheduler_ticks()`.

Parameters: None.

Returns: Returns elapsed normal 10 ms scheduler ticks.

Setup and ownership: Scheduler and direct RIT modes are compile-time exclusive. Exact RIT ownership also excludes the scheduler callback path.

Hardware/shared side effects: May power RIT and change PCLKSEL1, RICOMPVAL, RIMASK, RICOUNTER, RICTRL and NVIC state.

Copyable example:

```
uint32_t ticks = exam_rit_ticks();
```

Common mistake: Do not mix scheduler mode, direct RIT mode and an exact RIT handler. Only one model can own the RIT vector.

SysTick

exam_status_t exam_systick_every_ms(int milliseconds);

```
exam_status_t exam_systick_every_ms(int milliseconds);
```

Purpose: Configure periodic SysTick interrupts using an ordinary signed millisecond value. It is the beginner-facing wrapper for `systick_every_ms()`.

Parameters:

Declaration	Meaning
int milliseconds	Positive time in milliseconds.

Returns: EXAM_OK on success. Depending on the call, EXAM_RANGE, EXAM_INVALID, EXAM_BUSY or EXAM_NOT_READY reports a problem without guessing.

Setup and ownership: Use SysTick only when the paper names it. Tick counting requires the built-in SysTick handler; an exact handler must maintain its own state.

Hardware/shared side effects: May change SysTick CTRL, LOAD and VAL or read calibration/tick state.

Copyable example:

```
exam_systick_every_ms(10);
```

Common mistake: SysTick LOAD is 24-bit and stores reload-1 internally. Do not use the built-in tick counter while replacing its handler.

uint32_t exam_systick_ticks(void);

```
uint32_t exam_systick_ticks(void);
```

Purpose: Return the number of interrupts counted by the built-in SysTick handler. It is the beginner-facing wrapper for `systick_ticks()`.

Parameters: None.

Returns: Returns the built-in SysTick interrupt count.

Setup and ownership: Use SysTick only when the paper names it. Tick counting requires the built-in SysTick handler; an exact handler must maintain its own state.

Hardware/shared side effects: May change SysTick CTRL, LOAD and VAL or read calibration/tick state.

Copyable example:

```
uint32_t ticks = exam_systick_ticks();
```

Common mistake: SysTick LOAD is 24-bit and stores reload-1 internally. Do not use the built-in tick counter while replacing its handler.

ADC and potentiometer

exam_status_t exam_pot_start(void);

```
exam_status_t exam_pot_start(void);
```

Purpose: Initialize ADC channel 5 when first used, select software triggering, and request the first potentiometer conversion. It is the beginner-facing wrapper for `potentiometer_start()`.

Parameters: None.

Returns: EXAM_OK on success. Depending on the call, EXAM_RANGE, EXAM_INVALID, EXAM_BUSY or EXAM_NOT_READY reports a problem without guessing.

Setup and ownership: Fit JP12 for the board potentiometer. Cached reads require the built-in ADC_IRQHandler; exact ADC ownership returns BUSY for cached helpers.

Hardware/shared side effects: May power ADC, change pin selection and ADCR/ADINTEN, start conversions, and consume ISR-cached results.

Copyable example:

```
exam_pot_start();
```

Common mistake: Fit JP12. ADGDR is read once by the built-in handler; if you own ADC_IRQHandler, read it once yourself and do not use cached helpers.

exam_status_t exam_pot_read(int * value);

```
exam_status_t exam_pot_read(int * value);
```

Purpose: Return the next fresh potentiometer result and automatically request another conversion. It is the beginner-facing wrapper for potentiometer_read().

Parameters:

Declaration	Meaning
int * value	Address of an int that receives a fresh value from 0 through 4095.

Returns: EXAM_OK on success. Depending on the call, EXAM_RANGE, EXAM_INVALID, EXAM_BUSY or EXAM_NOT_READY reports a problem without guessing.

Setup and ownership: Fit JP12 for the board potentiometer. Cached reads require the built-in ADC_IRQHandler; exact ADC ownership returns BUSY for cached helpers.

Hardware/shared side effects: May power ADC, change pin selection and ADCR/ADINTEN, start conversions, and consume ISR-cached results.

Copyable example:

```
int value;
if (exam_pot_read(&value) == EXAM_OK) exam_led_write((uint8_t)(value >> 4));
```

Common mistake: Fit JP12. ADGDR is read once by the built-in handler; if you own ADC_IRQHandler, read it once yourself and do not use cached helpers.

exam_status_t exam_adc_read(uint8_t channel, uint16_t * value);

```
exam_status_t exam_adc_read(uint8_t channel, uint16_t * value);
```

Purpose: Copy only the latest cached 12-bit value for one ADC channel. It is the beginner-facing wrapper for adc_read_value().

Parameters:

Declaration	Meaning
uint8_t channel	ADC channel number 0 through 7.
uint16_t * value	Address of a uint16_t that receives the cached 12-bit result.

Returns: EXAM_OK on success. Depending on the call, EXAM_RANGE, EXAM_INVALID, EXAM_BUSY or EXAM_NOT_READY reports a problem without guessing.

Setup and ownership: Fit JP12 for the board potentiometer. Cached reads require the built-in ADC_IRQHandler; exact ADC ownership returns BUSY for cached helpers.

Hardware/shared side effects: May power ADC, change pin selection and ADCR/ADINTEN, start conversions, and consume ISR-cached results.

Copyable example:

```
uint16_t value;
if (exam_adc_read(5, &value) == EXAM_OK) { /* use value */ }
```

Common mistake: Fit JP12. ADGDR is read once by the built-in handler; if you own ADC_IRQHandler, read it once yourself and do not use cached helpers.

DAC and analog output

exam_status_t exam_dac_write(int value);

```
exam_status_t exam_dac_write(int value);
```

Purpose: Initialize AOUT when first used and write one validated 10-bit analog value. It is the beginner-facing wrapper for `speaker_write()`.

Parameters:

Declaration	Meaning
int value	Analog sample from 0 through 1023.

Returns: EXAM_OK on success. Depending on the call, EXAM_RANGE, EXAM_INVALID, EXAM_BUSY or EXAM_NOT_READY reports a problem without guessing.

Setup and ownership: Fit JP2 for speaker/analog testing. Simple write wrappers initialize AOUT on first use.

Hardware/shared side effects: May configure P0.26 as AOUT, write DACR, claim a timer, and update sample-playback state.

Copyable example:

```
exam_dac_write(512);
```

Common mistake: Fit JP2, keep values in 0..1023, keep waveform callbacks short, and do not claim a timer already used elsewhere.

exam_status_t exam_dac_percent(int percent);

```
exam_status_t exam_dac_percent(int percent);
```

Purpose: Convert 0 through 100 percent to a 10-bit DAC value and write it to AOUT. It is the beginner-facing wrapper for `speaker_write_percent()`.

Parameters:

Declaration	Meaning
int percent	Requested analog level from 0 through 100 percent.

Returns: EXAM_OK on success. Depending on the call, EXAM_RANGE, EXAM_INVALID, EXAM_BUSY or EXAM_NOT_READY reports a problem without guessing.

Setup and ownership: Fit JP2 for speaker/analog testing. Simple write wrappers initialize AOUT on first use.

Hardware/shared side effects: May configure P0.26 as AOUT, write DACR, claim a timer, and update sample-playback state.

Copyable example:

```
exam_dac_percent(50);
```

Common mistake: Fit JP2, keep values in 0..1023, keep waveform callbacks short, and do not claim a timer already used elsewhere.

void exam_dac_silence(void);

```
void exam_dac_silence(void);
```

Purpose: Write zero to DACR. It is the beginner-facing wrapper for `dac_silence()`.

Parameters: None.

Returns: No return value. Observe the documented hardware or shared-state effect.

Setup and ownership: Fit JP2 for speaker/analog testing. Simple write wrappers initialize AOUT on first use.

Hardware/shared side effects: May configure P0.26 as AOUT, write DACR, claim a timer, and update sample-playback state.

Copyable example:

```
exam_dac_silence();
```

Common mistake: Fit JP2, keep values in 0..1023, keep waveform callbacks short, and do not claim a timer already used elsewhere.

Interrupt communication

void exam_events_set(uint32_t bits);

```
void exam_events_set(uint32_t bits);
```

Purpose: Atomically OR event bits into the foreground event word. It is the beginner-facing wrapper for `event_flags_set()`.

Parameters:

Declaration	Meaning
uint32_t bits	Application-defined event bits; give each event a unique bit.

Returns: No return value. Observe the documented hardware or shared-state effect.

Setup and ownership: Use from callbacks and `exam_user_loop`. Keep critical sections short and restore the saved interrupt state exactly.

Hardware/shared side effects: Changes PRIMASK temporarily or atomically changes the internal event word.

Copyable example:

```
exam_events_set(EXAM_EVENT_TIMER);
```

Common mistake: Use volatile for interrupt-shared objects. Pass the saved PRIMASK back unchanged and never wait inside a critical section.

uint32_t exam_events_take(uint32_t mask);

```
uint32_t exam_events_take(uint32_t mask);
```

Purpose: Atomically return and clear only the requested event bits. It is the beginner-facing wrapper for `event_flags_take()`.

Parameters:

Declaration	Meaning
uint32_t mask	Bit mask interpreted by this function.

Returns: Returns the requested pending bits and clears those same bits.

Setup and ownership: Use from callbacks and `exam_user_loop`. Keep critical sections short and restore the saved interrupt state exactly.

Hardware/shared side effects: Changes PRIMASK temporarily or atomically changes the internal event word.

Copyable example:

```
if (exam_events_take(EXAM_EVENT_TIMER)) { /* foreground work */ }
```

Common mistake: Use volatile for interrupt-shared objects. Pass the saved PRIMASK back unchanged and never wait inside a critical section.

uint32_t exam_critical_enter(void);

```
uint32_t exam_critical_enter(void);
```

Purpose: Save PRIMASK, disable interrupts and issue a memory barrier before a short shared-data update. It is the beginner-facing wrapper for `critical_enter()`.

Parameters: None.

Returns: Returns the previous PRIMASK. Pass it unchanged to exam_critical_exit.

Setup and ownership: Use from callbacks and exam_user_loop. Keep critical sections short and restore the saved interrupt state exactly.

Hardware/shared side effects: Changes PRIMASK temporarily or atomically changes the internal event word.

Copyable example:

```
uint32_t key = exam_critical_enter();
shared_value = new_value;
exam_critical_exit(key);
```

Common mistake: Use volatile for interrupt-shared objects. Pass the saved PRIMASK back unchanged and never wait inside a critical section.

void exam_critical_exit(uint32_t saved_primask);

```
void exam_critical_exit(uint32_t saved_primask);
```

Purpose: Issue a memory barrier and restore the exact PRIMASK value saved by critical_enter. It is the beginner-facing wrapper for critical_exit().

Parameters:

Declaration	Meaning
uint32_t saved_primask	Exact value returned by exam_critical_enter.

Returns: No return value. Observe the documented hardware or shared-state effect.

Setup and ownership: Use from callbacks and exam_user_loop. Keep critical sections short and restore the saved interrupt state exactly.

Hardware/shared side effects: Changes PRIMASK temporarily or atomically changes the internal event word.

Copyable example:

```
exam_critical_exit(key); /* key came from exam_critical_enter() */
```

Common mistake: Use volatile for interrupt-shared objects. Pass the saved PRIMASK back unchanged and never wait inside a critical section.

Faults, SVC and hooks

uint32_t exam_self_test(void);

```
uint32_t exam_self_test(void);
```

Purpose: Run non-destructive configuration and range checks and return a mask of detected failures. It is the beginner-facing wrapper for board_self_test_run().

Parameters: None.

Returns: Returns an ORed SELF_TEST_* failure mask. Zero means no automatic failure detected.

Setup and ownership: Use only when the question requires this facility; normal peripheral answers do not need it.

Hardware/shared side effects: May change SCB fault controls, stacked SVC return registers, or diagnostic state.

Copyable example:

```
uint32_t failures = exam_self_test();
```

Common mistake: Use this only for its documented purpose and keep interrupt-context work short and bounded.

Precise driver and utility functions

Timers

board_status_t timer_every_ms(int timer, int milliseconds, timer_callback_t callback);

```
board_status_t timer_every_ms(int timer, int milliseconds, timer_callback_t callback);
```

Purpose: Configure MR0 as a repeating interrupt every requested number of milliseconds, attach a callback, reset the counter on each match, and start the selected timer.

Parameters:

Declaration	Meaning
int timer	Hardware timer number 0 through 3.
int milliseconds	Positive time in milliseconds.
timer_callback_t callback	Function called inside the matching interrupt; keep it short.

Returns: BOARD_OK on success. Depending on the call, BOARD_RANGE, BOARD_INVALID, BOARD_BUSY or BOARD_NOT_READY reports a problem without guessing.

Setup and ownership: The selected timer must use the template's handler. Owning that TIMERN_IRQHandler makes callback registration return BUSY. Other timers remain independent.

Hardware/shared side effects: May power the selected timer and change PCLKSEL, TCR, TC, PC, PR, CTCR, MCR, CCR, EMR, MR, IR and NVIC state.

Copyable example:

```
timer_every_ms(0, 1000, exam_timer_event);
```

Common mistake: Check PCLK, PR+1, units, timer number and W1C IR clearing. Never combine a callback helper with exact ownership of the same TIMERN vector.

board_status_t timer_every_hz(int timer, int hertz, timer_callback_t callback);

```
board_status_t timer_every_hz(int timer, int hertz, timer_callback_t callback);
```

Purpose: Configure MR0 for one callback per requested cycle frequency and start the selected timer.

Parameters:

Declaration	Meaning
int timer	Hardware timer number 0 through 3.
int hertz	Positive frequency in hertz.
timer_callback_t callback	Function called inside the matching interrupt; keep it short.

Returns: BOARD_OK on success. Depending on the call, BOARD_RANGE, BOARD_INVALID, BOARD_BUSY or BOARD_NOT_READY reports a problem without guessing.

Setup and ownership: The selected timer must use the template's handler. Owning that TIMERN_IRQHandler makes callback registration return BUSY. Other timers remain independent.

Hardware/shared side effects: May power the selected timer and change PCLKSEL, TCR, TC, PC, PR, CTCR, MCR, CCR, EMR, MR, IR and NVIC state.

Copyable example:

```
timer_every_hz(1, 100, exam_timer_event);
```

Common mistake: Check PCLK, PR+1, units, timer number and W1C IR clearing. Never combine a callback helper with exact ownership of the same TIMERN vector.

SysTick

board_status_t systick_every_ms(int milliseconds);

```
board_status_t systick_every_ms(int milliseconds);
```

Purpose: Configure periodic SysTick interrupts using an ordinary signed millisecond value.

Parameters:

Declaration	Meaning
int milliseconds	Positive time in milliseconds.

Returns: BOARD_OK on success. Depending on the call, BOARD_RANGE, BOARD_INVALID, BOARD_BUSY or BOARD_NOT_READY reports a problem without guessing.

Setup and ownership: Use SysTick only when the paper names it. Tick counting requires the built-in SysTick handler; an exact handler must maintain its own state.

Hardware/shared side effects: May change SysTick CTRL, LOAD and VAL or read calibration/tick state.

Copyable example:

```
systick_every_ms(10);
```

Common mistake: SysTick LOAD is 24-bit and stores reload-1 internally. Do not use the built-in tick counter while replacing its handler.

ADC and potentiometer

board_status_t potentiometer_start(void);

```
board_status_t potentiometer_start(void);
```

Purpose: Initialize ADC channel 5 when first used, select software triggering, and request the first potentiometer conversion.

Parameters: None.

Returns: BOARD_OK on success. Depending on the call, BOARD_RANGE, BOARD_INVALID, BOARD_BUSY or BOARD_NOT_READY reports a problem without guessing.

Setup and ownership: Fit JP12 for the board potentiometer. Cached reads require the built-in ADC_IRQHandler; exact ADC ownership returns BUSY for cached helpers.

Hardware/shared side effects: May power ADC, change pin selection and ADCR/ADINTEN, start conversions, and consume ISR-cached results.

Copyable example:

```
potentiometer_start();
```

Common mistake: Fit JP12. ADGDR is read once by the built-in handler; if you own ADC_IRQHandler, read it once yourself and do not use cached helpers.

board_status_t potentiometer_read(int * value);

```
board_status_t potentiometer_read(int * value);
```

Purpose: Return the next fresh potentiometer result and automatically request another conversion.

Parameters:

Declaration	Meaning
int * value	Address of an int that receives a fresh value from 0 through 4095.

Returns: BOARD_OK on success. Depending on the call, BOARD_RANGE, BOARD_INVALID, BOARD_BUSY or BOARD_NOT_READY reports a problem without guessing.

Setup and ownership: Fit JP12 for the board potentiometer. Cached reads require the built-in ADC_IRQHandler; exact ADC ownership returns BUSY for cached helpers.

Hardware/shared side effects: May power ADC, change pin selection and ADCR/ADINTEN, start conversions, and consume ISR-cached results.

Copyable example:

```
int value;
if (potentiometer_read(&value) == BOARD_OK) { /* use value */ }
```

Common mistake: Fit JP12. ADGDR is read once by the built-in handler; if you own ADC_IRQHandler, read it once yourself and do not use cached helpers.

DAC and analog output

board_status_t speaker_write(int value);

```
board_status_t speaker_write(int value);
```

Purpose: Initialize AOUT when first used and write one validated 10-bit analog value.

Parameters:

Declaration	Meaning
int value	Analog sample from 0 through 1023.

Returns: BOARD_OK on success. Depending on the call, BOARD_RANGE, BOARD_INVALID, BOARD_BUSY or BOARD_NOT_READY reports a problem without guessing.

Setup and ownership: Fit JP2 for speaker/analog testing. Simple write wrappers initialize AOUT on first use.

Hardware/shared side effects: May configure P0.26 as AOUT, write DACR, claim a timer, and update sample-playback state.

Copyable example:

```
speaker_write(512);
```

Common mistake: Fit JP2, keep values in 0..1023, keep waveform callbacks short, and do not claim a timer already used elsewhere.

board_status_t speaker_write_percent(int percent);

```
board_status_t speaker_write_percent(int percent);
```

Purpose: Convert 0 through 100 percent to a 10-bit DAC value and write it to AOUT.

Parameters:

Declaration	Meaning
int percent	Requested analog level from 0 through 100 percent.

Returns: BOARD_OK on success. Depending on the call, BOARD_RANGE, BOARD_INVALID, BOARD_BUSY or BOARD_NOT_READY reports a problem without guessing.

Setup and ownership: Fit JP2 for speaker/analog testing. Simple write wrappers initialize AOUT on first use.

Hardware/shared side effects: May configure P0.26 as AOUT, write DACR, claim a timer, and update sample-playback state.

Copyable example:

```
speaker_write_percent(50);
```

Common mistake: Fit JP2, keep values in 0..1023, keep waveform callbacks short, and do not claim a timer already used elsewhere.

External buttons

int int0_pressed(void);

```
int int0_pressed(void);
```

Purpose: Poll the active-low INT0 button and return a normal C true/false result.

Parameters: None.

Returns: Returns 1 when the tested condition is true and 0 otherwise.

Setup and ownership: Polling needs no interrupt. Inputs are active-low.

Hardware/shared side effects: May change PINSEL4, GPIO2 direction, EXTMODE, EXTPOLAR, EXTINT and the matching NVIC enable state.

Copyable example:

```
if (int0_pressed()) { /* held */ }
```

Common mistake: Do not add a blocking debounce delay. Callback confirmation needs the normal RIT scheduler; polling does not.

int key1_pressed(void);

```
int key1_pressed(void);
```

Purpose: Poll the active-low KEY1 button and return a normal C true/false result.

Parameters: None.

Returns: Returns 1 when the tested condition is true and 0 otherwise.

Setup and ownership: Polling needs no interrupt. Inputs are active-low.

Hardware/shared side effects: May change PINSEL4, GPIO2 direction, EXTMODE, EXTPOLAR, EXTINT and the matching NVIC enable state.

Copyable example:

```
if (key1_pressed()) { /* held */ }
```

Common mistake: Do not add a blocking debounce delay. Callback confirmation needs the normal RIT scheduler; polling does not.

int key2_pressed(void);

```
int key2_pressed(void);
```

Purpose: Poll the active-low KEY2 button and return a normal C true/false result.

Parameters: None.

Returns: Returns 1 when the tested condition is true and 0 otherwise.

Setup and ownership: Polling needs no interrupt. Inputs are active-low.

Hardware/shared side effects: May change PINSEL4, GPIO2 direction, EXTMODE, EXTPOLAR, EXTINT and the matching NVIC enable state.

Copyable example:

```
if (key2_pressed()) { /* held */ }
```

Common mistake: Do not add a blocking debounce delay. Callback confirmation needs the normal RIT scheduler; polling does not.

Joystick

int joystick_up_pressed(void);

```
int joystick_up_pressed(void);
```

Purpose: Return true while the joystick UP contact is active.

Parameters: None.

Returns: Returns 1 when the tested condition is true and 0 otherwise.

Setup and ownership: Polling reads GPIO directly. First-movement memory updates from the normal RIT service.

Hardware/shared side effects: Reads or configures GPIO1 and may update callback and first-movement state.

Copyable example:

```
if (joystick_up_pressed()) { /* up */ }
```

Common mistake: Direction values are masks. Test with &, not equality, because combinations can occur.

int joystick_down_pressed(void);

```
int joystick_down_pressed(void);
```

Purpose: Return true while the joystick DOWN contact is active.

Parameters: None.

Returns: Returns 1 when the tested condition is true and 0 otherwise.

Setup and ownership: Polling reads GPIO directly. First-movement memory updates from the normal RIT service.

Hardware/shared side effects: Reads or configures GPIO1 and may update callback and first-movement state.

Copyable example:

```
if (joystick_down_pressed()) { /* down */ }
```

Common mistake: Direction values are masks. Test with &, not equality, because combinations can occur.

int joystick_left_pressed(void);

```
int joystick_left_pressed(void);
```

Purpose: Return true while the joystick LEFT contact is active.

Parameters: None.

Returns: Returns 1 when the tested condition is true and 0 otherwise.

Setup and ownership: Polling reads GPIO directly. First-movement memory updates from the normal RIT service.

Hardware/shared side effects: Reads or configures GPIO1 and may update callback and first-movement state.

Copyable example:

```
if (joystick_left_pressed()) { /* left */ }
```

Common mistake: Direction values are masks. Test with &, not equality, because combinations can occur.

int joystick_right_pressed(void);

```
int joystick_right_pressed(void);
```

Purpose: Return true while the joystick RIGHT contact is active.

Parameters: None.

Returns: Returns 1 when the tested condition is true and 0 otherwise.

Setup and ownership: Polling reads GPIO directly. First-movement memory updates from the normal RIT service.

Hardware/shared side effects: Reads or configures GPIO1 and may update callback and first-movement state.

Copyable example:

```
if (joystick_right_pressed()) { /* right */ }
```

Common mistake: Direction values are masks. Test with &, not equality, because combinations can occur.

int joystick_button_pressed(void);

```
int joystick_button_pressed(void);
```

Purpose: Return true while the joystick centre/select contact is active.

Parameters: None.

Returns: Returns 1 when the tested condition is true and 0 otherwise.

Setup and ownership: Polling reads GPIO directly. First-movement memory updates from the normal RIT service.

Hardware/shared side effects: Reads or configures GPIO1 and may update callback and first-movement state.

Copyable example:

```
if (joystick_button_pressed()) { /* select */ }
```

Common mistake: Direction values are masks. Test with &, not equality, because combinations can occur.

Program control

void board_init(void);

```
void board_init(void);
```

Purpose: Initialize the system clock, LED GPIO and configurable fault support. The internal main calls it once before the answer code.

Parameters: None.

Returns: No return value. Observe the documented hardware or shared-state effect.

Setup and ownership: The internal main already calls this. Do not call it again in a normal answer.

Hardware/shared side effects: May change system clock setup, LED GPIO, SCB fault controls, or CPU sleep state as described.

Copyable example:

```
/* Called internally before exam_user_init(). */
```

Common mistake: Do not call it twice or assume that it initialized buttons, timers, RIT, joystick, ADC or DAC.

void board_idle(void);

```
void board_idle(void);
```

Purpose: Run the end of one foreground-loop iteration. It is a NOP by default and optionally executes WFI when explicitly configured.

Parameters: None.

Returns: No return value. Observe the documented hardware or shared-state effect.

Setup and ownership: The internal main already calls this. Do not call it again in a normal answer.

Hardware/shared side effects: May change system clock setup, LED GPIO, SCB fault controls, or CPU sleep state as described.

Copyable example:

```
/* Called internally after each exam_user_loop(). */
```

Common mistake: Enable WFI only when an enabled interrupt can wake the processor; the default NOP is safer for polling.

LED output

board_status_t led_write_number(uint8_t led_number, uint8_t on);

```
board_status_t led_write_number(uint8_t led_number, uint8_t on);
```

Purpose: Set or clear one LED selected by its printed LandTiger number 4 through 11.

Parameters:

Declaration	Meaning
uint8_t led_number	Printed LandTiger LED number 4 through 11.
uint8_t on	1 turns the selected LED on; 0 turns it off.

Returns: BOARD_OK on success. Depending on the call, BOARD_RANGE, BOARD_INVALID, BOARD_BUSY or BOARD_NOT_READY reports a problem without guessing.

Setup and ownership: No extra setup is required after the internal exam initialization. LED APIs use P2.0 through P2.7.

Hardware/shared side effects: Writes GPIO2 direction/set/clear state and updates the stored LED value.

Copyable example:

```
led_write_number(4, 1);
```

Common mistake: Do not confuse printed LED numbers 4..11 with legacy GPIO bit numbers 0..7.

board_status_t led_on(uint8_t led_number);

```
board_status_t led_on(uint8_t led_number);
```

Purpose: Turn on one LED selected by its printed number 4 through 11.

Parameters:

Declaration	Meaning
uint8_t led_number	Printed LandTiger LED number 4 through 11.

Returns: BOARD_OK on success. Depending on the call, BOARD_RANGE, BOARD_INVALID, BOARD_BUSY or BOARD_NOT_READY reports a problem without guessing.

Setup and ownership: No extra setup is required after the internal exam initialization. LED APIs use P2.0 through P2.7.

Hardware/shared side effects: Writes GPIO2 direction/set/clear state and updates the stored LED value.

Copyable example:

```
led_on(4);
```

Common mistake: Do not confuse printed LED numbers 4..11 with legacy GPIO bit numbers 0..7.

board_status_t led_off(uint8_t led_number);

```
board_status_t led_off(uint8_t led_number);
```

Purpose: Turn off one LED selected by its printed number 4 through 11.

Parameters:

Declaration	Meaning
uint8_t led_number	Printed LandTiger LED number 4 through 11.

Returns: BOARD_OK on success. Depending on the call, BOARD_RANGE, BOARD_INVALID, BOARD_BUSY or BOARD_NOT_READY reports a problem without guessing.

Setup and ownership: No extra setup is required after the internal exam initialization. LED APIs use P2.0 through P2.7.

Hardware/shared side effects: Writes GPIO2 direction/set/clear state and updates the stored LED value.

Copyable example:

```
led_off(4);
```

Common mistake: Do not confuse printed LED numbers 4..11 with legacy GPIO bit numbers 0..7.

void led_all_off(void);

```
void led_all_off(void);
```

Purpose: Turn off all eight LEDs in one operation.

Parameters: None.

Returns: No return value. Observe the documented hardware or shared-state effect.

Setup and ownership: No extra setup is required after the internal exam initialization. LED APIs use P2.0 through P2.7.

Hardware/shared side effects: Writes GPIO2 direction/set/clear state and updates the stored LED value.

Copyable example:

```
led_all_off();
```

Common mistake: Do not confuse printed LED numbers 4..11 with legacy GPIO bit numbers 0..7.

board_status_t led_write_mask(uint8_t mask);

```
board_status_t led_write_mask(uint8_t mask);
```

Purpose: Write the complete eight-bit LED row; bit 0 maps to P2.0 and printed LED4.

Parameters:

Declaration	Meaning
uint8_t mask	Bit mask interpreted by this function.

Returns: BOARD_OK on success. Depending on the call, BOARD_RANGE, BOARD_INVALID, BOARD_BUSY or BOARD_NOT_READY reports a problem without guessing.

Setup and ownership: No extra setup is required after the internal exam initialization. LED APIs use P2.0 through P2.7.

Hardware/shared side effects: Writes GPIO2 direction/set/clear state and updates the stored LED value.

Copyable example:

```
led_write_mask(0xA5);
```

Common mistake: Do not confuse printed LED numbers 4..11 with legacy GPIO bit numbers 0..7.

board_status_t led_toggle(uint8_t led_number);

```
board_status_t led_toggle(uint8_t led_number);
```

Purpose: Invert one LED selected by its printed number.

Parameters:

Declaration	Meaning
uint8_t led_number	Printed LandTiger LED number 4 through 11.

Returns: BOARD_OK on success. Depending on the call, BOARD_RANGE, BOARD_INVALID, BOARD_BUSY or BOARD_NOT_READY reports a problem without guessing.

Setup and ownership: No extra setup is required after the internal exam initialization. LED APIs use P2.0 through P2.7.

Hardware/shared side effects: Writes GPIO2 direction/set/clear state and updates the stored LED value.

Copyable example:

```
led_toggle(4);
```

Common mistake: Do not confuse printed LED numbers 4..11 with legacy GPIO bit numbers 0..7.

uint8_t led_read_mask(void);

```
uint8_t led_read_mask(void);
```

Purpose: Read the current eight LED output bits.

Parameters: None.

Returns: Returns the current low eight GPIO2 output bits.

Setup and ownership: No extra setup is required after the internal exam initialization. LED APIs use P2.0 through P2.7.

Hardware/shared side effects: Writes GPIO2 direction/set/clear state and updates the stored LED value.

Copyable example:

```
led_read_mask();
```

Common mistake: Do not confuse printed LED numbers 4..11 with legacy GPIO bit numbers 0..7.

void led_write8(uint8_t value);

```
void led_write8(uint8_t value);
```

Purpose: Write an eight-bit value using the familiar course-style LED helper.

Parameters:

Declaration	Meaning
uint8_t value	Low eight bits to display on the LED row.

Returns: No return value. Observe the documented hardware or shared-state effect.

Setup and ownership: No extra setup is required after the internal exam initialization. LED APIs use P2.0 through P2.7.

Hardware/shared side effects: Writes GPIO2 direction/set/clear state and updates the stored LED value.

Copyable example:

```
led_write8(0xA5);
```

Common mistake: Do not confuse printed LED numbers 4..11 with legacy GPIO bit numbers 0..7.

External buttons

void buttons_init(button_callback_t callback);

```
void buttons_init(button_callback_t callback);
```

Purpose: Configure INT0, KEY1 and KEY2 as falling-edge external interrupts and register an optional confirmed-event callback.

Parameters:

Declaration	Meaning
button_callback_t callback	Function called inside the matching interrupt; keep it short.

Returns: No return value. Observe the documented hardware or shared-state effect.

Setup and ownership: Call once in exam_user_init. For confirmed callbacks, start the normal RIT scheduler separately; polling does not need it.

Hardware/shared side effects: May change PINSEL4, GPIO2 direction, EXTMODE, EXTPOLAR, EXTINT and the matching NVIC enable state.

Copyable example:

```
rit_scheduler_start();
buttons_init(exam_button_event);
```

Common mistake: Do not add a blocking debounce delay. Callback confirmation needs the normal RIT scheduler; polling does not.

board_status_t button_irq_start(board_button_t button);

```
board_status_t button_irq_start(board_button_t button);
```

Purpose: Configure and enable one exact EINT vector without adding debounce, callbacks or RIT scheduling.

Parameters:

Declaration	Meaning
board_button_t button	One of INT0, KEY1 or KEY2 enum values.

Returns: BOARD_OK on success. Depending on the call, BOARD_RANGE, BOARD_INVALID, BOARD_BUSY or BOARD_NOT_READY reports a problem without guessing.

Setup and ownership: Enable the matching EXAM_OWN_EINTx_HANDLER switch, call this once, then provide that exact EINTx handler and clear EXTINT.

Hardware/shared side effects: May change PINSEL4, GPIO2 direction, EXTMODE, EXTPOLAR, EXTINT and the matching NVIC enable state.

Copyable example:

```
button_irq_start(BOARD_BUTTON_INT0);
```

Common mistake: Do not enable the exact handler without its ownership switch, and clear the matching EXTINT W1C bit in the handler.

void buttons_set_confirmation_ms(uint32_t milliseconds);

```
void buttons_set_confirmation_ms(uint32_t milliseconds);
```

Purpose: Change the software confirmation interval used by the external-button service.

Parameters:

Declaration	Meaning
uint32_t milliseconds	Confirmation interval. Use at least 10 and a multiple of 10 ms.

Returns: No return value. Observe the documented hardware or shared-state effect.

Setup and ownership: Polling needs no interrupt. Inputs are active-low.

Hardware/shared side effects: May change PINSEL4, GPIO2 direction, EXTMODE, EXTPOLAR, EXTINT and the matching NVIC enable state.

Copyable example:

```
buttons_set_confirmation_ms(50);
```

Common mistake: Do not add a blocking debounce delay. Callback confirmation needs the normal RIT scheduler; polling does not.

uint32_t buttons_pressed_mask(void);

```
uint32_t buttons_pressed_mask(void);
```

Purpose: Poll all three external buttons and return a bit mask of the buttons currently held.

Parameters: None.

Returns: Returns bits 0, 1 and 2 for INT0, KEY1 and KEY2 currently held.

Setup and ownership: Polling needs no interrupt. Inputs are active-low.

Hardware/shared side effects: May change PINSEL4, GPIO2 direction, EXTMODE, EXTPOLAR, EXTINT and the matching NVIC enable state.

Copyable example:

```
uint32_t held = buttons_pressed_mask();
```

Common mistake: Do not add a blocking debounce delay. Callback confirmation needs the normal RIT scheduler; polling does not.

uint8_t button_is_pressed(board_button_t button);

```
uint8_t button_is_pressed(board_button_t button);
```

Purpose: Poll one selected external button and return 1 while it is held.

Parameters:

Declaration	Meaning
board_button_t button	One of INT0, KEY1 or KEY2 enum values.

Returns: Returns 1 when the tested condition is true and 0 otherwise.

Setup and ownership: Polling needs no interrupt. Inputs are active-low.

Hardware/shared side effects: May change PINSEL4, GPIO2 direction, EXTMODE, EXTPOLAR, EXTINT and the matching NVIC enable state.

Copyable example:

```
if (button_is_pressed(BOARD_BUTTON_KEY1)) { /* pressed */ }
```

Common mistake: Do not add a blocking debounce delay. Callback confirmation needs the normal RIT scheduler; polling does not.

Joystick

void joystick_init(joystick_callback_t callback);

```
void joystick_init(joystick_callback_t callback);
```

Purpose: Configure the joystick inputs and register an optional change callback.

Parameters:

Declaration	Meaning
joystick_callback_t callback	Function called inside the matching interrupt; keep it short.

Returns: No return value. Observe the documented hardware or shared-state effect.

Setup and ownership: Call once in exam_user_init and start the normal RIT scheduler when callbacks or first-movement service are required.

Hardware/shared side effects: Reads or configures GPIO1 and may update callback and first-movement state.

Copyable example:

```
rit_scheduler_start();
joystick_init(exam_joystick_event);
```

Common mistake: Direction values are masks. Test with &, not equality, because combinations can occur.

uint32_t joystick_read(void);

```
uint32_t joystick_read(void);
```

Purpose: Return the current five-way joystick state as a mask, preserving combinations.

Parameters: None.

Returns: Returns an ORed mask of all currently active joystick directions.

Setup and ownership: Polling reads GPIO directly. First-movement memory updates from the normal RIT service.

Hardware/shared side effects: Reads or configures GPIO1 and may update callback and first-movement state.

Copyable example:

```
uint32_t directions = joystick_read();
```

Common mistake: Direction values are masks. Test with &, not equality, because combinations can occur.

uint8_t joystick_is_pressed(uint32_t direction);

```
uint8_t joystick_is_pressed(uint32_t direction);
```

Purpose: Test whether every direction bit requested by the caller is currently held.

Parameters:

Declaration	Meaning
uint32_t direction	One joystick direction mask or an ORed combination.

Returns: Returns 1 when the tested condition is true and 0 otherwise.

Setup and ownership: Polling reads GPIO directly. First-movement memory updates from the normal RIT service.

Hardware/shared side effects: Reads or configures GPIO1 and may update callback and first-movement state.

Copyable example:

```
if (joystick_is_pressed(JOYSTICK_UP)) { /* up */ }
```

Common mistake: Direction values are masks. Test with &, not equality, because combinations can occur.

uint32_t joystick_first_movement(void);

```
uint32_t joystick_first_movement(void);
```

Purpose: Return the first nonzero joystick movement remembered after the last reset.

Parameters: None.

Returns: Returns the remembered first direction mask, or 0 if none has been serviced.

Setup and ownership: Polling reads GPIO directly. First-movement memory updates from the normal RIT service.

Hardware/shared side effects: Reads or configures GPIO1 and may update callback and first-movement state.

Copyable example:

```
uint32_t first = joystick_first_movement();
```

Common mistake: Direction values are masks. Test with &, not equality, because combinations can occur.

void joystick_reset_first_movement(void);

```
void joystick_reset_first_movement(void);
```

Purpose: Clear first-movement memory before starting a new trial or round.

Parameters: None.

Returns: No return value. Observe the documented hardware or shared-state effect.

Setup and ownership: Polling reads GPIO directly. First-movement memory updates from the normal RIT service.

Hardware/shared side effects: Reads or configures GPIO1 and may update callback and first-movement state.

Copyable example:

```
joystick_reset_first_movement();
```

Common mistake: Direction values are masks. Test with &, not equality, because combinations can occur.

Timers

board_status_t timer_set_clock_divider(uint8_t timer, uint8_t divider);

```
board_status_t timer_set_clock_divider(uint8_t timer, uint8_t divider);
```

Purpose: Select the peripheral-clock divider for Timer 0, 1, 2 or 3 while that timer is stopped.

Parameters:

Declaration	Meaning
uint8_t timer	Hardware timer number 0 through 3.
uint8_t divider	Peripheral-clock divider 1, 2, 4 or 8.

Returns: BOARD_OK on success. Depending on the call, BOARD_RANGE, BOARD_INVALID, BOARD_BUSY or BOARD_NOT_READY reports a problem without guessing.

Setup and ownership: Use timer 0 through 3. Configure its clock, prescaler and mode before starting when the question grades those values.

Hardware/shared side effects: May power the selected timer and change PCLKSEL, TCR, TC, PC, PR, CTCR, MCR, CCR, EMR, MR, IR and NVIC state.

Copyable example:

```
timer_set_clock_divider(0, 4);
```

Common mistake: Check PCLK, PR+1, units, timer number and W1C IR clearing. Never combine a callback helper with exact ownership of the same TIMERN vector.

uint32_t timer_peripheral_clock_hz(uint8_t timer);

```
uint32_t timer_peripheral_clock_hz(uint8_t timer);
```

Purpose: Calculate and return the selected timer's peripheral clock in hertz.

Parameters:

Declaration	Meaning
uint8_t timer	Hardware timer number 0 through 3.

Returns: Returns the timer PCLK in hertz, or 0 for an invalid timer.

Setup and ownership: Use timer 0 through 3. Configure its clock, prescaler and mode before starting when the question grades those values.

Hardware/shared side effects: May power the selected timer and change PCLKSEL, TCR, TC, PC, PR, CTCR, MCR, CCR, EMR, MR, IR and NVIC state.

Copyable example:

```
uint32_t pclk = timer_peripheral_clock_hz(0);
```

Common mistake: Check PCLK, PR+1, units, timer number and W1C IR clearing. Never combine a callback helper with exact ownership of the same TIMERN vector.

board_status_t timer_set_prescaler(uint8_t timer, uint32_t prescaler);

```
board_status_t timer_set_prescaler(uint8_t timer, uint32_t prescaler);
```

Purpose: Write PR so TC increments once every PR+1 peripheral-clock ticks.

Parameters:

Declaration	Meaning
uint8_t timer	Hardware timer number 0 through 3.
uint32_t prescaler	PR value. TC divides its input clock by prescaler+1.

Returns: BOARD_OK on success. Depending on the call, BOARD_RANGE, BOARD_INVALID, BOARD_BUSY or BOARD_NOT_READY reports a problem without guessing.

Setup and ownership: Use timer 0 through 3. Configure its clock, prescaler and mode before starting when the question grades those values.

Hardware/shared side effects: May power the selected timer and change PCLKSEL, TCR, TC, PC, PR, CTCR, MCR, CCR, EMR, MR, IR and NVIC state.

Copyable example:

```
timer_set_prescaler(0, 0);
```

Common mistake: Check PCLK, PR+1, units, timer number and W1C IR clearing. Never combine a callback helper with exact ownership of the same TIMERN vector.

uint32_t timer_counter_clock_hz(uint8_t timer);

```
uint32_t timer_counter_clock_hz(uint8_t timer);
```

Purpose: Return the effective TC rate after the PCLK divider and PR+1 prescaler.

Parameters:

Declaration	Meaning
uint8_t timer	Hardware timer number 0 through 3.

Returns: Returns the effective TC clock in hertz, or 0 for an invalid timer.

Setup and ownership: Use timer 0 through 3. Configure its clock, prescaler and mode before starting when the question grades those values.

Hardware/shared side effects: May power the selected timer and change PCLKSEL, TCR, TC, PC, PR, CTCR, MCR, CCR, EMR, MR, IR and NVIC state.

Copyable example:

```
uint32_t tc_hz = timer_counter_clock_hz(0);
```

Common mistake: Check PCLK, PR+1, units, timer number and W1C IR clearing. Never combine a callback helper with exact ownership of the same TIMERN vector.

board_status_t timer_calculate_match_for_frequency(uint8_t timer, uint32_t frequency_hz, uint32_t events_per_cycle, uint32_t * match_value);

```
board_status_t timer_calculate_match_for_frequency(
    uint8_t timer,
    uint32_t frequency_hz,
    uint32_t events_per_cycle,
    uint32_t * match_value
);
```

Purpose: Calculate the nearest whole MR tick count for a frequency and number of events per cycle without programming the timer.

Parameters:

Declaration	Meaning
uint8_t timer	Hardware timer number 0 through 3.
uint32_t frequency_hz	Requested cycle frequency in hertz.
uint32_t events_per_cycle	Number of timer events per complete logical cycle; usually 1, 2 for LED toggles, or a waveform sample count.
uint32_t * match_value	Address that receives the calculated timer-tick value.

Returns: BOARD_OK on success. Depending on the call, BOARD_RANGE, BOARD_INVALID, BOARD_BUSY or BOARD_NOT_READY reports a problem without guessing.

Setup and ownership: Use timer 0 through 3. Configure its clock, prescaler and mode before starting when the question grades those values.

Hardware/shared side effects: May power the selected timer and change PCLKSEL, TCR, TC, PC, PR, CTCR, MCR, CCR, EMR, MR, IR and NVIC state.

Copyable example:

```
uint32_t mr;
if (timer_calculate_match_for_frequency(1, 440, 45, &mr) == BOARD_OK) { /* inspect mr */ }
```

Common mistake: Check PCLK, PR+1, units, timer number and W1C IR clearing. Never combine a callback helper with exact ownership of the same TIMERN vector.

board_status_t timer_configure_frequency(uint8_t timer, uint8_t match, uint32_t frequency_hz, uint32_t events_per_cycle, uint32_t actions);

```
board_status_t timer_configure_frequency(
    uint8_t timer,
    uint8_t match,
    uint32_t frequency_hz,
    uint32_t events_per_cycle,
    uint32_t actions
);
```

Purpose: Calculate and program one match register from a requested frequency and events-per-cycle value.

Parameters:

Declaration	Meaning
uint8_t timer	Hardware timer number 0 through 3.
uint8_t match	Match channel 0 through 3.
uint32_t frequency_hz	Requested cycle frequency in hertz.
uint32_t events_per_cycle	Number of timer events per complete logical cycle; usually 1, 2 for LED toggles, or a waveform sample count.
uint32_t actions	ORed interrupt, reset and stop action bits.

Returns: BOARD_OK on success. Depending on the call, BOARD_RANGE, BOARD_INVALID, BOARD_BUSY or BOARD_NOT_READY reports a problem without guessing.

Setup and ownership: Use timer 0 through 3. Configure its clock, prescaler and mode before starting when the question grades those values.

Hardware/shared side effects: May power the selected timer and change PCLKSEL, TCR, TC, PC, PR, CTCR, MCR, CCR, EMR, MR, IR and NVIC state.

Copyable example:

```
timer_configure_frequency(0, 0, 2, 2, TIMER_ACTION_INTERRUPT | TIMER_ACTION_RESET);
```

Common mistake: Check PCLK, PR+1, units, timer number and W1C IR clearing. Never combine a callback helper with exact ownership of the same TIMERN vector.

board_status_t timer_start_periodic_interrupt_hz(uint8_t timer, uint32_t frequency_hz, uint32_t events_per_cycle, timer_callback_t callback);

```
board_status_t timer_start_periodic_interrupt_hz(
    uint8_t timer,
    uint32_t frequency_hz,
    uint32_t events_per_cycle,
    timer_callback_t callback
);
```

Purpose: Configure a complete periodic MR0 interrupt in hertz, attach a callback, clear stale flags and start the timer.

Parameters:

Declaration	Meaning
uint8_t timer	Hardware timer number 0 through 3.
uint32_t frequency_hz	Requested cycle frequency in hertz.
uint32_t events_per_cycle	Number of timer events per complete logical cycle; usually 1, 2 for LED toggles, or a waveform sample count.
timer_callback_t callback	Function called inside the matching interrupt; keep it short.

Returns: BOARD_OK on success. Depending on the call, BOARD_RANGE, BOARD_INVALID, BOARD_BUSY or BOARD_NOT_READY reports a problem without guessing.

Setup and ownership: The selected timer must use the template's handler. Owning that TIMERN_IRQHandler makes callback registration return BUSY. Other timers remain independent.

Hardware/shared side effects: May power the selected timer and change PCLKSEL, TCR, TC, PC, PR, CTCR, MCR, CCR, EMR, MR, IR and NVIC state.

Copyable example:

```
timer_start_periodic_interrupt_hz(0, 100, 1, exam_timer_event);
```

Common mistake: Check PCLK, PR+1, units, timer number and W1C IR clearing. Never combine a callback helper with exact ownership of the same TIMERN vector.

board_status_t timer_start_periodic_interrupt_ms(uint8_t timer, uint32_t period_ms, timer_callback_t callback);

```
board_status_t timer_start_periodic_interrupt_ms(
    uint8_t timer,
    uint32_t period_ms,
    timer_callback_t callback
);
```

Purpose: Configure a complete periodic MR0 interrupt in milliseconds, attach a callback, clear stale flags and start the timer.

Parameters:

Declaration	Meaning
uint8_t timer	Hardware timer number 0 through 3.
uint32_t period_ms	Positive period in milliseconds.
timer_callback_t callback	Function called inside the matching interrupt; keep it short.

Returns: BOARD_OK on success. Depending on the call, BOARD_RANGE, BOARD_INVALID, BOARD_BUSY or BOARD_NOT_READY reports a problem without guessing.

Setup and ownership: The selected timer must use the template's handler. Owning that TIMERN_IRQHandler makes callback registration return BUSY. Other timers remain independent.

Hardware/shared side effects: May power the selected timer and change PCLKSEL, TCR, TC, PC, PR, CTCR, MCR, CCR, EMR, MR, IR and NVIC state.

Copyable example:

```
timer_start_periodic_interrupt_ms(1, 250, exam_timer_event);
```

Common mistake: Check PCLK, PR+1, units, timer number and W1C IR clearing. Never combine a callback helper with exact ownership of the same TIMERN vector.

uint8_t timer_match_occurred(uint32_t pending_flags, uint8_t match);

```
uint8_t timer_match_occurred(uint32_t pending_flags, uint8_t match);
```

Purpose: Test callback pending flags for MR0, MR1, MR2 or MR3.

Parameters:

Declaration	Meaning
uint32_t pending_flags	Snapshot of timer IR flags supplied to a callback.
uint8_t match	Match channel 0 through 3.

Returns: Returns 1 when the tested condition is true and 0 otherwise.

Setup and ownership: Use timer 0 through 3. Configure its clock, prescaler and mode before starting when the question grades those values.

Hardware/shared side effects: May power the selected timer and change PCLKSEL, TCR, TC, PC, PR, CTCR, MCR, CCR, EMR, MR, IR and NVIC state.

Copyable example:

```
if (timer_match_occurred(flags, 0)) { /* MR0 */ }
```

Common mistake: Check PCLK, PR+1, units, timer number and W1C IR clearing. Never combine a callback helper with exact ownership of the same TIMERN vector.

uint8_t timer_capture_occurred(uint32_t pending_flags, uint8_t capture);

```
uint8_t timer_capture_occurred(uint32_t pending_flags, uint8_t capture);
```

Purpose: Test callback pending flags for CR0 or CR1.

Parameters:

Declaration	Meaning
uint32_t pending_flags	Snapshot of timer IR flags supplied to a callback.
uint8_t capture	Capture channel 0 or 1.

Returns: Returns 1 when the tested condition is true and 0 otherwise.

Setup and ownership: Use timer 0 through 3. Configure its clock, prescaler and mode before starting when the question grades those values.

Hardware/shared side effects: May power the selected timer and change PCLKSEL, TCR, TC, PC, PR, CTCR, MCR, CCR, EMR, MR, IR and NVIC state.

Copyable example:

```
if (timer_capture_occurred(flags, 1)) { /* CR1 */ }
```

Common mistake: Check PCLK, PR+1, units, timer number and W1C IR clearing. Never combine a callback helper with exact ownership of the same TIMERN vector.

board_status_t timer_configure_match(uint8_t timer, uint8_t match, uint32_t value, uint32_t actions);

```
board_status_t timer_configure_match(
    uint8_t timer,
    uint8_t match,
    uint32_t value,
    uint32_t actions
);
```

Purpose: Write MR0 through MR3 and program the matching MCR interrupt, reset and stop actions.

Parameters:

Declaration	Meaning
uint8_t timer	Hardware timer number 0 through 3.

Declaration	Meaning
uint8_t match	Match channel 0 through 3.
uint32_t value	Exact match value in timer-counter ticks.
uint32_t actions	ORed interrupt, reset and stop action bits.

Returns: BOARD_OK on success. Depending on the call, BOARD_RANGE, BOARD_INVALID, BOARD_BUSY or BOARD_NOT_READY reports a problem without guessing.

Setup and ownership: Use timer 0 through 3. Configure its clock, prescaler and mode before starting when the question grades those values.

Hardware/shared side effects: May power the selected timer and change PCLKSEL, TCR, TC, PC, PR, CTCR, MCR, CCR, EMR, MR, IR and NVIC state.

Copyable example:

```
timer_configure_match(0, 0, 25000000, TIMER_ACTION_INTERRUPT | TIMER_ACTION_RESET);
```

Common mistake: Check PCLK, PR+1, units, timer number and W1C IR clearing. Never combine a callback helper with exact ownership of the same TIMERN vector.

board_status_t timer_configure_capture(uint8_t timer, uint8_t capture, timer_capture_edge_t edge, uint8_t interrupt_enable);

```
board_status_t timer_configure_capture(
    uint8_t timer,
    uint8_t capture,
    timer_capture_edge_t edge,
    uint8_t interrupt_enable
);
```

Purpose: Configure one capture input for rising, falling or both edges and optionally enable its interrupt.

Parameters:

Declaration	Meaning
uint8_t timer	Hardware timer number 0 through 3.
uint8_t capture	Capture channel 0 or 1.
timer_capture_edge_t edge	Rising, falling or both-edge capture selection.
uint8_t interrupt_enable	1 enables the interrupt; 0 leaves it disabled.

Returns: BOARD_OK on success. Depending on the call, BOARD_RANGE, BOARD_INVALID, BOARD_BUSY or BOARD_NOT_READY reports a problem without guessing.

Setup and ownership: Use timer 0 through 3. Configure its clock, prescaler and mode before starting when the question grades those values.

Hardware/shared side effects: May power the selected timer and change PCLKSEL, TCR, TC, PC, PR, CTCR, MCR, CCR, EMR, MR, IR and NVIC state.

Copyable example:

```
timer_configure_capture(0, 1, CAPTURE_RISING, 1);
```

Common mistake: Check PCLK, PR+1, units, timer number and W1C IR clearing. Never combine a callback helper with exact ownership of the same TIMERN vector.

board_status_t timer_read_capture(uint8_t timer, uint8_t capture, uint32_t * value);

```
board_status_t timer_read_capture(uint8_t timer, uint8_t capture, uint32_t * value);
```

Purpose: Copy the latest CR0 or CR1 captured counter value to caller-provided storage.

Parameters:

Declaration	Meaning
uint8_t timer	Hardware timer number 0 through 3.

Declaration	Meaning
uint8_t capture	Capture channel 0 or 1.
uint32_t * value	Address of a uint32_t that receives the captured count.

Returns: BOARD_OK on success. Depending on the call, BOARD_RANGE, BOARD_INVALID, BOARD_BUSY or BOARD_NOT_READY reports a problem without guessing.

Setup and ownership: Use timer 0 through 3. Configure its clock, prescaler and mode before starting when the question grades those values.

Hardware/shared side effects: May power the selected timer and change PCLKSEL, TCR, TC, PC, PR, CTCR, MCR, CCR, EMR, MR, IR and NVIC state.

Copyable example:

```
uint32_t captured;
timer_read_capture(0, 1, &captured);
```

Common mistake: Check PCLK, PR+1, units, timer number and W1C IR clearing. Never combine a callback helper with exact ownership of the same TIMERN vector.

board_status_t timer_set_counter_mode(uint8_t timer, timer_counter_mode_t mode, uint8_t capture_input);

```
board_status_t timer_set_counter_mode(
    uint8_t timer,
    timer_counter_mode_t mode,
    uint8_t capture_input
);
```

Purpose: Choose timer mode or external counter mode and select CAP0 or CAP1 as the counting input.

Parameters:

Declaration	Meaning
uint8_t timer	Hardware timer number 0 through 3.
timer_counter_mode_t mode	Timer mode or one of the three external-counter edge modes.
uint8_t capture_input	CAP input 0 or 1.

Returns: BOARD_OK on success. Depending on the call, BOARD_RANGE, BOARD_INVALID, BOARD_BUSY or BOARD_NOT_READY reports a problem without guessing.

Setup and ownership: Use timer 0 through 3. Configure its clock, prescaler and mode before starting when the question grades those values.

Hardware/shared side effects: May power the selected timer and change PCLKSEL, TCR, TC, PC, PR, CTCR, MCR, CCR, EMR, MR, IR and NVIC state.

Copyable example:

```
timer_set_counter_mode(0, TIMER_MODE_COUNTER_RISING, 0);
```

Common mistake: Check PCLK, PR+1, units, timer number and W1C IR clearing. Never combine a callback helper with exact ownership of the same TIMERN vector.

board_status_t timer_configure_external_match(uint8_t timer, uint8_t match, timer_external_match_t action, uint8_t initial_state);

```
board_status_t timer_configure_external_match(
    uint8_t timer,
    uint8_t match,
    timer_external_match_t action,
    uint8_t initial_state
);
```

Purpose: Program the MAT output action and initial output state for one match channel.

Parameters:

Declaration	Meaning
uint8_t timer	Hardware timer number 0 through 3.
uint8_t match	Match channel 0 through 3.
timer_external_match_t action	External-match action: do nothing, clear, set or toggle.
uint8_t initial_state	Initial MAT output bit, 0 or 1.

Returns: BOARD_OK on success. Depending on the call, BOARD_RANGE, BOARD_INVALID, BOARD_BUSY or BOARD_NOT_READY reports a problem without guessing.

Setup and ownership: Use timer 0 through 3. Configure its clock, prescaler and mode before starting when the question grades those values.

Hardware/shared side effects: May power the selected timer and change PCLKSEL, TCR, TC, PC, PR, CTCR, MCR, CCR, EMR, MR, IR and NVIC state.

Copyable example:

```
timer_configure_external_match(1, 0, EXT_MATCH_TOGGLE, 0);
```

Common mistake: Check PCLK, PR+1, units, timer number and W1C IR clearing. Never combine a callback helper with exact ownership of the same TIMERN vector.

board_status_t timer_start(uint8_t timer);

```
board_status_t timer_start(uint8_t timer);
```

Purpose: Start or resume one hardware timer by setting TCR enable.

Parameters:

Declaration	Meaning
uint8_t timer	Hardware timer number 0 through 3.

Returns: BOARD_OK on success. Depending on the call, BOARD_RANGE, BOARD_INVALID, BOARD_BUSY or BOARD_NOT_READY reports a problem without guessing.

Setup and ownership: Use timer 0 through 3. Configure its clock, prescaler and mode before starting when the question grades those values.

Hardware/shared side effects: May power the selected timer and change PCLKSEL, TCR, TC, PC, PR, CTCR, MCR, CCR, EMR, MR, IR and NVIC state.

Copyable example:

```
timer_start(0);
```

Common mistake: Check PCLK, PR+1, units, timer number and W1C IR clearing. Never combine a callback helper with exact ownership of the same TIMERN vector.

board_status_t timer_stop(uint8_t timer);

```
board_status_t timer_stop(uint8_t timer);
```

Purpose: Stop one hardware timer without clearing its current TC value.

Parameters:

Declaration	Meaning
uint8_t timer	Hardware timer number 0 through 3.

Returns: BOARD_OK on success. Depending on the call, BOARD_RANGE, BOARD_INVALID, BOARD_BUSY or BOARD_NOT_READY reports a problem without guessing.

Setup and ownership: Use timer 0 through 3. Configure its clock, prescaler and mode before starting when the question grades those values.

Hardware/shared side effects: May power the selected timer and change PCLKSEL, TCR, TC, PC, PR, CTCR, MCR, CCR, EMR, MR, IR and NVIC state.

Copyable example:

```
timer_stop(0);
```

Common mistake: Check PCLK, PR+1, units, timer number and W1C IR clearing. Never combine a callback helper with exact ownership of the same TIMERN vector.

board_status_t timer_reset(uint8_t timer);

```
board_status_t timer_reset(uint8_t timer);
```

Purpose: Pulse the timer reset bit, clear TC and PC, and leave the timer stopped.

Parameters:

Declaration	Meaning
uint8_t timer	Hardware timer number 0 through 3.

Returns: BOARD_OK on success. Depending on the call, BOARD_RANGE, BOARD_INVALID, BOARD_BUSY or BOARD_NOT_READY reports a problem without guessing.

Setup and ownership: Use timer 0 through 3. Configure its clock, prescaler and mode before starting when the question grades those values.

Hardware/shared side effects: May power the selected timer and change PCLKSEL, TCR, TC, PC, PR, CTCR, MCR, CCR, EMR, MR, IR and NVIC state.

Copyable example:

```
timer_reset(0);
```

Common mistake: Check PCLK, PR+1, units, timer number and W1C IR clearing. Never combine a callback helper with exact ownership of the same TIMERN vector.

board_status_t timer_set_callback(uint8_t timer, timer_callback_t callback);

```
board_status_t timer_set_callback(uint8_t timer, timer_callback_t callback);
```

Purpose: Register the callback used by the template's handler for one timer.

Parameters:

Declaration	Meaning
uint8_t timer	Hardware timer number 0 through 3.
timer_callback_t callback	Function called inside the matching interrupt; keep it short.

Returns: BOARD_OK on success. Depending on the call, BOARD_RANGE, BOARD_INVALID, BOARD_BUSY or BOARD_NOT_READY reports a problem without guessing.

Setup and ownership: The selected timer must use the template's handler. Owning that TIMERN_IRQHandler makes callback registration return BUSY. Other timers remain independent.

Hardware/shared side effects: May power the selected timer and change PCLKSEL, TCR, TC, PC, PR, CTCR, MCR, CCR, EMR, MR, IR and NVIC state.

Copyable example:

```
timer_set_callback(0, exam_timer_event);
```

Common mistake: Check PCLK, PR+1, units, timer number and W1C IR clearing. Never combine a callback helper with exact ownership of the same TIMERN vector.

uint32_t timer_read_counter(uint8_t timer);

```
uint32_t timer_read_counter(uint8_t timer);
```

Purpose: Read the current TC value of a timer, including a free-running timer.

Parameters:

Declaration	Meaning
uint8_t timer	Hardware timer number 0 through 3.

Returns: Returns TC, or 0 for an invalid timer.

Setup and ownership: Use timer 0 through 3. Configure its clock, prescaler and mode before starting when the question grades those values.

Hardware/shared side effects: May power the selected timer and change PCLKSEL, TCR, TC, PC, PR, CTCR, MCR, CCR, EMR, MR, IR and NVIC state.

Copyable example:

```
uint32_t elapsed = timer_read_counter(1);
```

Common mistake: Check PCLK, PR+1, units, timer number and W1C IR clearing. Never combine a callback helper with exact ownership of the same TIMERN vector.

RIT

board_status_t rit_scheduler_start(void);

```
board_status_t rit_scheduler_start(void);
```

Purpose: Configure and start the normal 10 ms RIT service used by confirmed buttons, joystick callbacks and the user hook.

Parameters: None.

Returns: BOARD_OK on success. Depending on the call, BOARD_RANGE, BOARD_INVALID, BOARD_BUSY or BOARD_NOT_READY reports a problem without guessing.

Setup and ownership: Scheduler and direct RIT modes are compile-time exclusive. Exact RIT ownership also excludes the scheduler callback path.

Hardware/shared side effects: May power RIT and change PCLKSEL1, RICOMPVAL, RIMASK, RICOUNTER, RICTRL and NVIC state.

Copyable example:

```
rit_scheduler_start();
```

Common mistake: Do not mix scheduler mode, direct RIT mode and an exact RIT handler. Only one model can own the RIT vector.

void rit_scheduler_stop(void);

```
void rit_scheduler_stop(void);
```

Purpose: Stop the RIT counter used by the normal scheduler.

Parameters: None.

Returns: No return value. Observe the documented hardware or shared-state effect.

Setup and ownership: Scheduler and direct RIT modes are compile-time exclusive. Exact RIT ownership also excludes the scheduler callback path.

Hardware/shared side effects: May power RIT and change PCLKSEL1, RICOMPVAL, RIMASK, RICOUNTER, RICTRL and NVIC state.

Copyable example:

```
rit_scheduler_stop();
```

Common mistake: Do not mix scheduler mode, direct RIT mode and an exact RIT handler. Only one model can own the RIT vector.

uint32_t rit_scheduler_ticks(void);

```
uint32_t rit_scheduler_ticks(void);
```

Purpose: Return the number of 10 ms scheduler interrupts handled since reset.

Parameters: None.

Returns: Returns elapsed normal 10 ms scheduler ticks.

Setup and ownership: Scheduler and direct RIT modes are compile-time exclusive. Exact RIT ownership also excludes the scheduler callback path.

Hardware/shared side effects: May power RIT and change PCLKSEL1, RICOMPVAL, RIMASK, RICOUNTER, RICTRL and NVIC state.

Copyable example:

```
uint32_t ticks = rit_scheduler_ticks();
```

Common mistake: Do not mix scheduler mode, direct RIT mode and an exact RIT handler. Only one model can own the RIT vector.

board_status_t rit_raw_configure(uint32_t compare, uint32_t mask, uint8_t clear_on_match);

```
board_status_t rit_raw_configure(
    uint32_t compare,
    uint32_t mask,
    uint8_t clear_on_match
);
```

Purpose: Program RICOMPVAL, RIMASK, RICOUNTER and clear-on-match behavior in compile-time direct RIT mode.

Parameters:

Declaration	Meaning
uint32_t compare	RIT compare value written to RICOMPVAL.
uint32_t mask	RIMASK value. Bits set to 1 are ignored during compare.
uint8_t clear_on_match	1 clears RICOUNTER on a match; 0 lets it continue.

Returns: BOARD_OK on success. Depending on the call, BOARD_RANGE, BOARD_INVALID, BOARD_BUSY or BOARD_NOT_READY reports a problem without guessing.

Setup and ownership: Scheduler and direct RIT modes are compile-time exclusive. Exact RIT ownership also excludes the scheduler callback path.

Hardware/shared side effects: May power RIT and change PCLKSEL1, RICOMPVAL, RIMASK, RICOUNTER, RICTRL and NVIC state.

Copyable example:

```
rit_raw_configure(1000000, 0, 1);
```

Common mistake: Do not mix scheduler mode, direct RIT mode and an exact RIT handler. Only one model can own the RIT vector.

board_status_t rit_raw_start(void);

```
board_status_t rit_raw_start(void);
```

Purpose: Enable counting in direct RIT mode.

Parameters: None.

Returns: BOARD_OK on success. Depending on the call, BOARD_RANGE, BOARD_INVALID, BOARD_BUSY or BOARD_NOT_READY reports a problem without guessing.

Setup and ownership: Scheduler and direct RIT modes are compile-time exclusive. Exact RIT ownership also excludes the scheduler callback path.

Hardware/shared side effects: May power RIT and change PCLKSEL1, RICOMPVAL, RIMASK, RICOUNTER, RICTRL and NVIC state.

Copyable example:

```
rit_raw_start();
```

Common mistake: Do not mix scheduler mode, direct RIT mode and an exact RIT handler. Only one model can own the RIT vector.

void rit_raw_stop(void);

```
void rit_raw_stop(void);
```

Purpose: Disable RIT counting without erasing the current counter.

Parameters: None.

Returns: No return value. Observe the documented hardware or shared-state effect.

Setup and ownership: Scheduler and direct RIT modes are compile-time exclusive. Exact RIT ownership also excludes the scheduler callback path.

Hardware/shared side effects: May power RIT and change PCLKSEL1, RICOMPVAL, RIMASK, RICOUNTER, RICTRL and NVIC state.

Copyable example:

```
rit_raw_stop();
```

Common mistake: Do not mix scheduler mode, direct RIT mode and an exact RIT handler. Only one model can own the RIT vector.

uint32_t rit_raw_read(void);

```
uint32_t rit_raw_read(void);
```

Purpose: Read the current RICOUNTER value.

Parameters: None.

Returns: Returns the current RICOUNTER value.

Setup and ownership: Scheduler and direct RIT modes are compile-time exclusive. Exact RIT ownership also excludes the scheduler callback path.

Hardware/shared side effects: May power RIT and change PCLKSEL1, RICOMPVAL, RIMASK, RICOUNTER, RICTRL and NVIC state.

Copyable example:

```
uint32_t count = rit_raw_read();
```

Common mistake: Do not mix scheduler mode, direct RIT mode and an exact RIT handler. Only one model can own the RIT vector.

SysTick

board_status_t systick_configure(uint32_t reload, uint8_t periodic, uint8_t interrupt_enable);

```
board_status_t systick_configure(
    uint32_t reload,
    uint8_t periodic,
    uint8_t interrupt_enable
);
```

Purpose: Stop SysTick, program its 24-bit reload, clear the current value, select one-shot or periodic behavior, and optionally enable its interrupt.

Parameters:

Declaration	Meaning
uint32_t reload	SysTick reload from 1 through 0xFFFFFFFF.
uint8_t periodic	1 repeats; 0 disables SysTick after the first interrupt.
uint8_t interrupt_enable	1 enables the interrupt; 0 leaves it disabled.

Returns: BOARD_OK on success. Depending on the call, BOARD_RANGE, BOARD_INVALID, BOARD_BUSY or BOARD_NOT_READY reports a problem without guessing.

Setup and ownership: Use SysTick only when the paper names it. Tick counting requires the built-in SysTick handler; an exact handler must maintain its own state.

Hardware/shared side effects: May change SysTick CTRL, LOAD and VAL or read calibration/tick state.

Copyable example:

```
systick_configure(99999, 1, 1);
```

Common mistake: SysTick LOAD is 24-bit and stores reload-1 internally. Do not use the built-in tick counter while replacing its handler.

board_status_t systick_start_periodic_ms(uint32_t period_ms);

```
board_status_t systick_start_periodic_ms(uint32_t period_ms);
```

Purpose: Calculate a reload from milliseconds, select the processor clock and start periodic SysTick interrupts.

Parameters:

Declaration	Meaning
uint32_t period_ms	Positive period in milliseconds.

Returns: BOARD_OK on success. Depending on the call, BOARD_RANGE, BOARD_INVALID, BOARD_BUSY or BOARD_NOT_READY reports a problem without guessing.

Setup and ownership: Use SysTick only when the paper names it. Tick counting requires the built-in SysTick handler; an exact handler must maintain its own state.

Hardware/shared side effects: May change SysTick CTRL, LOAD and VAL or read calibration/tick state.

Copyable example:

```
systick_start_periodic_ms(10);
```

Common mistake: SysTick LOAD is 24-bit and stores reload-1 internally. Do not use the built-in tick counter while replacing its handler.

board_status_t systick_set_clock_source(uint8_t processor_clock);

```
board_status_t systick_set_clock_source(uint8_t processor_clock);
```

Purpose: Select the processor clock or reference clock as the SysTick source while preserving the enable state.

Parameters:

Declaration	Meaning
uint8_t processor_clock	1 selects the processor clock; 0 selects the reference path.

Returns: BOARD_OK on success. Depending on the call, BOARD_RANGE, BOARD_INVALID, BOARD_BUSY or BOARD_NOT_READY reports a problem without guessing.

Setup and ownership: Use SysTick only when the paper names it. Tick counting requires the built-in SysTick handler; an exact handler must maintain its own state.

Hardware/shared side effects: May change SysTick CTRL, LOAD and VAL or read calibration/tick state.

Copyable example:

```
systick_set_clock_source(1);
```

Common mistake: SysTick LOAD is 24-bit and stores reload-1 internally. Do not use the built-in tick counter while replacing its handler.

uint32_t systick_calibration_value(void);

```
uint32_t systick_calibration_value(void);
```

Purpose: Return the hardware TENMS calibration field.

Parameters: None.

Returns: Returns the 24-bit TENMS calibration field.

Setup and ownership: Use SysTick only when the paper names it. Tick counting requires the built-in SysTick handler; an exact handler must maintain its own state.

Hardware/shared side effects: May change SysTick CTRL, LOAD and VAL or read calibration/tick state.

Copyable example:

```
uint32_t ten_ms = systick_calibration_value();
```

Common mistake: SysTick LOAD is 24-bit and stores reload-1 internally. Do not use the built-in tick counter while replacing its handler.

uint8_t systick_has_precise_calibration(void);

```
uint8_t systick_has_precise_calibration(void);
```

Purpose: Report whether the SysTick calibration exists and is not marked skewed.

Parameters: None.

Returns: Returns 1 when the tested condition is true and 0 otherwise.

Setup and ownership: Use SysTick only when the paper names it. Tick counting requires the built-in SysTick handler; an exact handler must maintain its own state.

Hardware/shared side effects: May change SysTick CTRL, LOAD and VAL or read calibration/tick state.

Copyable example:

```
if (systick_has_precise_calibration()) { /* calibrated */ }
```

Common mistake: SysTick LOAD is 24-bit and stores reload-1 internally. Do not use the built-in tick counter while replacing its handler.

uint32_t systick_ticks(void);

```
uint32_t systick_ticks(void);
```

Purpose: Return the number of interrupts counted by the built-in SysTick handler.

Parameters: None.

Returns: Returns the built-in SysTick interrupt count.

Setup and ownership: Use SysTick only when the paper names it. Tick counting requires the built-in SysTick handler; an exact handler must maintain its own state.

Hardware/shared side effects: May change SysTick CTRL, LOAD and VAL or read calibration/tick state.

Copyable example:

```
uint32_t now = systick_ticks();
```

Common mistake: SysTick LOAD is 24-bit and stores reload-1 internally. Do not use the built-in tick counter while replacing its handler.

ADC and potentiometer

board_status_t adc_init(uint8_t channel, uint32_t peripheral_clock_hz);

```
board_status_t adc_init(uint8_t channel, uint32_t peripheral_clock_hz);
```

Purpose: Power the ADC, select a channel and pin, choose a safe CLKDIV at or below 13 MHz, and enable the ADC interrupt.

Parameters:

Declaration	Meaning
uint8_t channel	ADC channel number 0 through 7.
uint32_t peripheral_clock_hz	Actual ADC peripheral clock before CLKDIV, in hertz.

Returns: BOARD_OK on success. Depending on the call, BOARD_RANGE, BOARD_INVALID, BOARD_BUSY or BOARD_NOT_READY reports a problem without guessing.

Setup and ownership: Power/configure the ADC before conversions. If you own ADC_IRQHandler, read ADGDR once and decode/store the result yourself.

Hardware/shared side effects: May power ADC, change pin selection and ADCR/ADINTEN, start conversions, and consume ISR-cached results.

Copyable example:

```
adc_init(5, SystemFrequency / 4);
```

Common mistake: Fit JP12. ADGDR is read once by the built-in handler; if you own ADC_IRQHandler, read it once yourself and do not use cached helpers.

board_status_t adc_configure_trigger(adc_trigger_t trigger, uint8_t falling_edge);

```
board_status_t adc_configure_trigger(adc_trigger_t trigger, uint8_t falling_edge);
```

Purpose: Select software, external-pin or timer-match conversion triggering and choose the active edge.

Parameters:

Declaration	Meaning
adc_trigger_t trigger	Software, external-pin or timer-match ADC trigger enum.
uint8_t falling_edge	1 selects falling edge; 0 selects rising edge.

Returns: BOARD_OK on success. Depending on the call, BOARD_RANGE, BOARD_INVALID, BOARD_BUSY or BOARD_NOT_READY reports a problem without guessing.

Setup and ownership: Power/configure the ADC before conversions. If you own ADC_IRQHandler, read ADGDR once and decode/store the result yourself.

Hardware/shared side effects: May power ADC, change pin selection and ADCR/ADINTEN, start conversions, and consume ISR-cached results.

Copyable example:

```
adc_configure_trigger(ADC_TRIGGER_SOFTWARE, 0);
```

Common mistake: Fit JP12. ADGDR is read once by the built-in handler; if you own ADC_IRQHandler, read it once yourself and do not use cached helpers.

board_status_t adc_start_conversion(void);

```
board_status_t adc_start_conversion(void);
```

Purpose: Start one software-triggered ADC conversion.

Parameters: None.

Returns: BOARD_OK on success. Depending on the call, BOARD_RANGE, BOARD_INVALID, BOARD_BUSY or BOARD_NOT_READY reports a problem without guessing.

Setup and ownership: Power/configure the ADC before conversions. If you own ADC_IRQHandler, read ADGDR once and decode/store the result yourself.

Hardware/shared side effects: May power ADC, change pin selection and ADCR/ADINTEN, start conversions, and consume ISR-cached results.

Copyable example:

```
adc_start_conversion();
```

Common mistake: Fit JP12. ADGDR is read once by the built-in handler; if you own ADC_IRQHandler, read it once yourself and do not use cached helpers.

board_status_t adc_start_burst(uint32_t channel_mask);

```
board_status_t adc_start_burst(uint32_t channel_mask);
```

Purpose: Enable continuous burst conversion for one or more channels.

Parameters:

Declaration	Meaning
uint32_t channel_mask	Bits 0 through 7 select ADC burst channels.

Returns: BOARD_OK on success. Depending on the call, BOARD_RANGE, BOARD_INVALID, BOARD_BUSY or BOARD_NOT_READY reports a problem without guessing.

Setup and ownership: Power/configure the ADC before conversions. If you own ADC_IRQHandler, read ADGDR once and decode/store the result yourself.

Hardware/shared side effects: May power ADC, change pin selection and ADCR/ADINTEN, start conversions, and consume ISR-cached results.

Copyable example:

```
adc_start_burst(1u << 5);
```

Common mistake: Fit JP12. ADGDR is read once by the built-in handler; if you own ADC_IRQHandler, read it once yourself and do not use cached helpers.

void adc_stop_burst(void);

```
void adc_stop_burst(void);
```

Purpose: Disable ADC burst mode.

Parameters: None.

Returns: No return value. Observe the documented hardware or shared-state effect.

Setup and ownership: Power/configure the ADC before conversions. If you own ADC_IRQHandler, read ADGDR once and decode/store the result yourself.

Hardware/shared side effects: May power ADC, change pin selection and ADCR/ADINTEN, start conversions, and consume ISR-cached results.

Copyable example:

```
adc_stop_burst();
```

Common mistake: Fit JP12. ADGDR is read once by the built-in handler; if you own ADC_IRQHandler, read it once yourself and do not use cached helpers.

board_status_t adc_read_channel(uint8_t channel, adc_sample_t * sample);

```
board_status_t adc_read_channel(uint8_t channel, adc_sample_t * sample);
```

Purpose: Copy the latest ISR-cached raw and decoded ADC sample for one channel.

Parameters:

Declaration	Meaning
uint8_t channel	ADC channel number 0 through 7.
adc_sample_t * sample	Address of adc_sample_t that receives raw, value, channel, DONE and overrun fields.

Returns: BOARD_OK on success. Depending on the call, BOARD_RANGE, BOARD_INVALID, BOARD_BUSY or BOARD_NOT_READY reports a problem without guessing.

Setup and ownership: Fit JP12 for the board potentiometer. Cached reads require the built-in ADC_IRQHandler; exact ADC ownership returns BUSY for cached helpers.

Hardware/shared side effects: May power ADC, change pin selection and ADCR/ADINTEN, start conversions, and consume ISR-cached results.

Copyable example:

```
adc_sample_t sample;
if (adc_read_channel(5, &sample) == BOARD_OK) { /* sample.value */ }
```

Common mistake: Fit JP12. ADGDR is read once by the built-in handler; if you own ADC_IRQHandler, read it once yourself and do not use cached helpers.

board_status_t adc_read_value(uint8_t channel, uint16_t * value);

```
board_status_t adc_read_value(uint8_t channel, uint16_t * value);
```

Purpose: Copy only the latest cached 12-bit value for one ADC channel.

Parameters:

Declaration	Meaning
uint8_t channel	ADC channel number 0 through 7.
uint16_t * value	Address of a uint16_t that receives the cached 12-bit result.

Returns: BOARD_OK on success. Depending on the call, BOARD_RANGE, BOARD_INVALID, BOARD_BUSY or BOARD_NOT_READY reports a problem without guessing.

Setup and ownership: Fit JP12 for the board potentiometer. Cached reads require the built-in ADC_IRQHandler; exact ADC ownership returns BUSY for cached helpers.

Hardware/shared side effects: May power ADC, change pin selection and ADCR/ADINTEN, start conversions, and consume ISR-cached results.

Copyable example:

```
uint16_t value;
adc_read_value(5, &value);
```

Common mistake: Fit JP12. ADGDR is read once by the built-in handler; if you own ADC_IRQHandler, read it once yourself and do not use cached helpers.

uint32_t adc_clock_hz(void);

```
uint32_t adc_clock_hz(void);
```

Purpose: Return the actual ADC clock selected during initialization.

Parameters: None.

Returns: Returns the calculated ADC clock in hertz; it is 0 before initialization.

Setup and ownership: Power/configure the ADC before conversions. If you own ADC_IRQHandler, read ADGDR once and decode/store the result yourself.

Hardware/shared side effects: May power ADC, change pin selection and ADCR/ADINTEN, start conversions, and consume ISR-cached results.

Copyable example:

```
uint32_t adc_hz = adc_clock_hz();
```

Common mistake: Fit JP12. ADGDR is read once by the built-in handler; if you own ADC_IRQHandler, read it once yourself and do not use cached helpers.

DAC and analog output

board_status_t dac_init(void);

```
board_status_t dac_init(void);
```

Purpose: Configure P0.26 as AOUT and initialize the DAC output to zero.

Parameters: None.

Returns: BOARD_OK on success. Depending on the call, BOARD_RANGE, BOARD_INVALID, BOARD_BUSY or BOARD_NOT_READY reports a problem without guessing.

Setup and ownership: Fit JP2 for speaker/analog testing. Simple write wrappers initialize AOUT on first use.

Hardware/shared side effects: May configure P0.26 as AOUT, write DACR, claim a timer, and update sample-playback state.

Copyable example:

```
dac_init();
```

Common mistake: Fit JP2, keep values in 0..1023, keep waveform callbacks short, and do not claim a timer already used elsewhere.

board_status_t dac_write(uint16_t value);

```
board_status_t dac_write(uint16_t value);
```

Purpose: Write one validated 10-bit sample to DACR.

Parameters:

Declaration	Meaning
uint16_t value	Ten-bit DAC sample from 0 through 1023.

Returns: BOARD_OK on success. Depending on the call, BOARD_RANGE, BOARD_INVALID, BOARD_BUSY or BOARD_NOT_READY reports a problem without guessing.

Setup and ownership: Fit JP2 for speaker/analog testing. Simple write wrappers initialize AOUT on first use.

Hardware/shared side effects: May configure P0.26 as AOUT, write DACR, claim a timer, and update sample-playback state.

Copyable example:

```
dac_write(512);
```

Common mistake: Fit JP2, keep values in 0..1023, keep waveform callbacks short, and do not claim a timer already used elsewhere.

void dac_silence(void);

```
void dac_silence(void);
```

Purpose: Write zero to DACR.

Parameters: None.

Returns: No return value. Observe the documented hardware or shared-state effect.

Setup and ownership: Fit JP2 for speaker/analog testing. Simple write wrappers initialize AOUT on first use.

Hardware/shared side effects: May configure P0.26 as AOUT, write DACR, claim a timer, and update sample-playback state.

Copyable example:

```
dac_silence();
```

Common mistake: Fit JP2, keep values in 0..1023, keep waveform callbacks short, and do not claim a timer already used elsewhere.

board_status_t dac_validate_update_rate(uint32_t update_hz);

```
board_status_t dac_validate_update_rate(uint32_t update_hz);
```

Purpose: Check that a requested sample-update rate is nonzero and no greater than 1 MHz.

Parameters:

Declaration	Meaning
uint32_t update_hz	DAC sample-update rate in hertz; maximum 1,000,000.

Returns: BOARD_OK on success. Depending on the call, BOARD_RANGE, BOARD_INVALID, BOARD_BUSY or BOARD_NOT_READY reports a problem without guessing.

Setup and ownership: Fit JP2 for speaker/analog testing. Simple write wrappers initialize AOUT on first use.

Hardware/shared side effects: May configure P0.26 as AOUT, write DACR, claim a timer, and update sample-playback state.

Copyable example:

```
if (dac_validate_update_rate(44100) == BOARD_OK) { /* valid */ }
```

Common mistake: Fit JP2, keep values in 0..1023, keep waveform callbacks short, and do not claim a timer already used elsewhere.

board_status_t dac_play_samples(const uint16_t * samples, uint32_t count, uint32_t update_hz, uint8_t timer);

```
board_status_t dac_play_samples(
    const uint16_t * samples,
    uint32_t count,
    uint32_t update_hz,
    uint8_t timer
);
```

Purpose: Claim one timer and play a sample table from its MR0 interrupt until every sample has been written.

Parameters:

Declaration	Meaning
const uint16_t * samples	Pointer to a persistent table of 10-bit DAC samples.
uint32_t count	Number of entries in the sample table.
uint32_t update_hz	DAC sample-update rate in hertz; maximum 1,000,000.
uint8_t timer	Hardware timer number 0 through 3.

Returns: BOARD_OK on success. Depending on the call, BOARD_RANGE, BOARD_INVALID, BOARD_BUSY or BOARD_NOT_READY reports a problem without guessing.

Setup and ownership: Fit JP2. The selected timer and its template handler must be free; the sample table must remain valid until playback ends.

Hardware/shared side effects: May configure P0.26 as AOUT, write DACR, claim a timer, and update sample-playback state.

Copyable example:

```
static const uint16_t wave[] = {0, 512, 1023, 512};
dac_play_samples(wave, 4, 4000, 2);
```

Common mistake: Fit JP2, keep values in 0..1023, keep waveform callbacks short, and do not claim a timer already used elsewhere.

Interrupt communication

uint32_t critical_enter(void);

```
uint32_t critical_enter(void);
```

Purpose: Save PRIMASK, disable interrupts and issue a memory barrier before a short shared-data update.

Parameters: None.

Returns: Returns the previous PRIMASK. Pass it unchanged to critical_exit.

Setup and ownership: Use from callbacks and exam_user_loop. Keep critical sections short and restore the saved interrupt state exactly.

Hardware/shared side effects: Changes PRIMASK temporarily or atomically changes the internal event word.

Copyable example:

```
uint32_t key = critical_enter();
shared_value = new_value;
critical_exit(key);
```

Common mistake: Use volatile for interrupt-shared objects. Pass the saved PRIMASK back unchanged and never wait inside a critical section.

void critical_exit(uint32_t previous_primask);

```
void critical_exit(uint32_t previous_primask);
```

Purpose: Issue a memory barrier and restore the exact PRIMASK value saved by critical_enter.

Parameters:

Declaration	Meaning
uint32_t previous_primask	Exact value returned by critical_enter.

Returns: No return value. Observe the documented hardware or shared-state effect.

Setup and ownership: Use from callbacks and exam_user_loop. Keep critical sections short and restore the saved interrupt state exactly.

Hardware/shared side effects: Changes PRIMASK temporarily or atomically changes the internal event word.

Copyable example:

```
critical_exit(key); /* key came from critical_enter() */
```

Common mistake: Use volatile for interrupt-shared objects. Pass the saved PRIMASK back unchanged and never wait inside a critical section.

void event_flags_set(uint32_t flags);

```
void event_flags_set(uint32_t flags);
```

Purpose: Atomically OR event bits into the foreground event word.

Parameters:

Declaration	Meaning
uint32_t flags	Bit mask supplied or consumed by this function.

Returns: No return value. Observe the documented hardware or shared-state effect.

Setup and ownership: Use from callbacks and exam_user_loop. Keep critical sections short and restore the saved interrupt state exactly.

Hardware/shared side effects: Changes PRIMASK temporarily or atomically changes the internal event word.

Copyable example:

```
event_flags_set(1u << 0);
```

Common mistake: Use volatile for interrupt-shared objects. Pass the saved PRIMASK back unchanged and never wait inside a critical section.

uint32_t event_flags_take(uint32_t mask);

```
uint32_t event_flags_take(uint32_t mask);
```

Purpose: Atomically return and clear only the requested event bits.

Parameters:

Declaration	Meaning
uint32_t mask	Bit mask interpreted by this function.

Returns: Returns the requested pending bits and clears those same bits.

Setup and ownership: Use from callbacks and exam_user_loop. Keep critical sections short and restore the saved interrupt state exactly.

Hardware/shared side effects: Changes PRIMASK temporarily or atomically changes the internal event word.

Copyable example:

```
if (event_flags_take(1u << 0)) { /* foreground work */ }
```

Common mistake: Use volatile for interrupt-shared objects. Pass the saved PRIMASK back unchanged and never wait inside a critical section.

Faults, SVC and hooks

void fault_traps_configure(void);

```
void fault_traps_configure(void);
```

Purpose: Apply divide-by-zero, unaligned-access and configurable-fault choices from exam_config.h.

Parameters: None.

Returns: No return value. Observe the documented hardware or shared-state effect.

Setup and ownership: Use only when the question requires this facility; normal peripheral answers do not need it.

Hardware/shared side effects: May change SCB fault controls, stacked SVC return registers, or diagnostic state.

Copyable example:

```
/* Called internally by board_init(). */
```

Common mistake: Use this only for its documented purpose and keep interrupt-context work short and bounded.

void svc_dispatch(uint8_t service_number, svc_context_t * context);

```
void svc_dispatch(uint8_t service_number, svc_context_t * context);
```

Purpose: Handle a decoded SVC number in C using the hardware-stacked register context; the default implementation is weak.

Parameters:

Declaration	Meaning
uint8_t service_number	Decoded eight-bit SVC immediate number.
svc_context_t * context	Pointer to the hardware-stacked register frame.

Returns: No return value. Observe the documented hardware or shared-state effect.

Setup and ownership: Use only when the question requires this facility; normal peripheral answers do not need it.

Hardware/shared side effects: May change SCB fault controls, stacked SVC return registers, or diagnostic state.

Copyable example:

```
void svc_dispatch(uint8_t n, svc_context_t *c)
{
    if (n == 0) c->r0 = c->r0 + 1;
}
```

Common mistake: Modify only the required stacked return registers. Do not treat context->pc as the SVC instruction address without subtracting 2.

Convenience and control

void * board_direct_register(const char * name);

```
void * board_direct_register(const char * name);
```

Purpose: Return the base address of one named LPC1768 peripheral as an advanced escape hatch when a typed helper is not available.

Parameters:

Declaration	Meaning
const char * name	One supported peripheral name such as TIMER0, RIT, ADC or DAC.

Returns: Returns a void pointer to the requested peripheral block, or a null pointer for an unsupported name.

Setup and ownership: Use only when the question requires this facility; normal peripheral answers do not need it.

Hardware/shared side effects: Changes only the state documented for this wrapper and its underlying precise function.

Copyable example:

```
LPC_TIM_TypeDef *timer = (LPC_TIM_TypeDef *)board_direct_register("TIMER0");
```

Common mistake: Use this only for its documented purpose and keep interrupt-context work short and bounded.

Faults, SVC and hooks

void exam_user_10ms_hook(void);

```
void exam_user_10ms_hook(void);
```

Purpose: Provide the answer's optional short action on each normal 10 ms RIT scheduler tick.

Parameters: None.

Returns: No return value. Observe the documented hardware or shared-state effect.

Setup and ownership: Use only when the question requires this facility; normal peripheral answers do not need it.

Hardware/shared side effects: May change SCB fault controls, stacked SVC return registers, or diagnostic state.

Copyable example:

```
void exam_user_10ms_hook(void)
{
    /* one short bounded action */
}
```

Common mistake: Use this only for its documented purpose and keep interrupt-context work short and bounded.

uint32_t board_self_test_run(void);

```
uint32_t board_self_test_run(void);
```

Purpose: Run non-destructive configuration and range checks and return a mask of detected failures.

Parameters: None.

Returns: Returns an ORed SELF_TEST_* failure mask. Zero means no automatic failure detected.

Setup and ownership: Use only when the question requires this facility; normal peripheral answers do not need it.

Hardware/shared side effects: May change SCB fault controls, stacked SVC return registers, or diagnostic state.

Copyable example:

```
uint32_t failures = board_self_test_run();
```

Common mistake: Use this only for its documented purpose and keep interrupt-context work short and bounded.

Familiar course-compatible functions

LED output

void LED_init(void);

```
void LED_init(void);
```

Purpose: Configure P2.0 through P2.7 as GPIO outputs, clear all LEDs and reset the legacy led_value variable.

Parameters: None.

Returns: No return value. Observe the documented hardware or shared-state effect.

Setup and ownership: No extra setup is required after the internal exam initialization. LED APIs use P2.0 through P2.7.

Hardware/shared side effects: Writes GPIO2 direction/set/clear state and updates the stored LED value.

Copyable example:

```
LED_init();
```

Common mistake: Do not confuse printed LED numbers 4..11 with legacy GPIO bit numbers 0..7.

void LED_deinit(void);

```
void LED_deinit(void);
```

Purpose: Return P2.0 through P2.7 to input direction.

Parameters: None.

Returns: No return value. Observe the documented hardware or shared-state effect.

Setup and ownership: No extra setup is required after the internal exam initialization. LED APIs use P2.0 through P2.7.

Hardware/shared side effects: Writes GPIO2 direction/set/clear state and updates the stored LED value.

Copyable example:

```
LED_deinit();
```

Common mistake: Do not confuse printed LED numbers 4..11 with legacy GPIO bit numbers 0..7.

void LED_On(unsigned int bit_number);

```
void LED_On(unsigned int bit_number);
```

Purpose: Set one LED GPIO bit using legacy bit numbering 0 through 7.

Parameters:

Declaration	Meaning
unsigned int bit_number	Legacy LED GPIO bit 0 through 7, not printed number 4 through 11.

Returns: No return value. Observe the documented hardware or shared-state effect.

Setup and ownership: No extra setup is required after the internal exam initialization. LED APIs use P2.0 through P2.7.

Hardware/shared side effects: Writes GPIO2 direction/set/clear state and updates the stored LED value.

Copyable example:

```
LED_On(0);
```

Common mistake: Do not confuse printed LED numbers 4..11 with legacy GPIO bit numbers 0..7.

void LED_Off(unsigned int bit_number);

```
void LED_Off(unsigned int bit_number);
```

Purpose: Clear one LED GPIO bit using legacy bit numbering 0 through 7.

Parameters:

Declaration	Meaning
unsigned int bit_number	Legacy LED GPIO bit 0 through 7, not printed number 4 through 11.

Returns: No return value. Observe the documented hardware or shared-state effect.

Setup and ownership: No extra setup is required after the internal exam initialization. LED APIs use P2.0 through P2.7.

Hardware/shared side effects: Writes GPIO2 direction/set/clear state and updates the stored LED value.

Copyable example:

```
LED_Off(0);
```

Common mistake: Do not confuse printed LED numbers 4..11 with legacy GPIO bit numbers 0..7.

void LED_Out(unsigned int value);

```
void LED_Out(unsigned int value);
```

Purpose: Write the low eight bits of one value to the LED row.

Parameters:

Declaration	Meaning
unsigned int value	Low eight bits to display on the LED row.

Returns: No return value. Observe the documented hardware or shared-state effect.

Setup and ownership: No extra setup is required after the internal exam initialization. LED APIs use P2.0 through P2.7.

Hardware/shared side effects: Writes GPIO2 direction/set/clear state and updates the stored LED value.

Copyable example:

```
LED_Out(0xA5);
```

Common mistake: Do not confuse printed LED numbers 4..11 with legacy GPIO bit numbers 0..7.

External buttons

void BUTTON_init(void);

```
void BUTTON_init(void);
```

Purpose: Run the familiar course button initialization with no callback registered.

Parameters: None.

Returns: No return value. Observe the documented hardware or shared-state effect.

Setup and ownership: Polling needs no interrupt. Inputs are active-low.

Hardware/shared side effects: May change PINSEL4, GPIO2 direction, EXTMODE, EXTPOLAR, EXTINT and the matching NVIC enable state.

Copyable example:

```
BUTTON_init();
```

Common mistake: Do not add a blocking debounce delay. Callback confirmation needs the normal RIT scheduler; polling does not.

Timers

uint32_t init_timer(uint8_t timer_num, uint32_t interval);

```
uint32_t init_timer(uint8_t timer_num, uint32_t interval);
```

Purpose: Configure MR0 of Timer 0 through 3 for interrupt plus reset using an interval already expressed in timer ticks.

Parameters:

Declaration	Meaning
uint8_t timer_num	Hardware timer number 0 through 3.

Declaration	Meaning
uint32_t interval	MR0 interval already calculated in timer ticks.

Returns: Returns 1 when configuration succeeds and 0 when the timer number or configuration is invalid.

Setup and ownership: Use timer 0 through 3. Configure its clock, prescaler and mode before starting when the question grades those values.

Hardware/shared side effects: May power the selected timer and change PCLKSEL, TCR, TC, PC, PR, CTCR, MCR, CCR, EMR, MR, IR and NVIC state.

Copyable example:

```
if (init_timer(0, 25000000)) enable_timer(0);
```

Common mistake: Check PCLK, PR+1, units, timer number and W1C IR clearing. Never combine a callback helper with exact ownership of the same TIMERN vector.

void enable_timer(uint8_t timer_num);

```
void enable_timer(uint8_t timer_num);
```

Purpose: Start the selected timer using the familiar course name.

Parameters:

Declaration	Meaning
uint8_t timer_num	Hardware timer number 0 through 3.

Returns: No return value. Observe the documented hardware or shared-state effect.

Setup and ownership: Use timer 0 through 3. Configure its clock, prescaler and mode before starting when the question grades those values.

Hardware/shared side effects: May power the selected timer and change PCLKSEL, TCR, TC, PC, PR, CTCR, MCR, CCR, EMR, MR, IR and NVIC state.

Copyable example:

```
enable_timer(0);
```

Common mistake: Check PCLK, PR+1, units, timer number and W1C IR clearing. Never combine a callback helper with exact ownership of the same TIMERN vector.

void disable_timer(uint8_t timer_num);

```
void disable_timer(uint8_t timer_num);
```

Purpose: Stop the selected timer using the familiar course name.

Parameters:

Declaration	Meaning
uint8_t timer_num	Hardware timer number 0 through 3.

Returns: No return value. Observe the documented hardware or shared-state effect.

Setup and ownership: Use timer 0 through 3. Configure its clock, prescaler and mode before starting when the question grades those values.

Hardware/shared side effects: May power the selected timer and change PCLKSEL, TCR, TC, PC, PR, CTCR, MCR, CCR, EMR, MR, IR and NVIC state.

Copyable example:

```
disable_timer(0);
```

Common mistake: Check PCLK, PR+1, units, timer number and W1C IR clearing. Never combine a callback helper with exact ownership of the same TIMERN vector.

void reset_timer(uint8_t timer_num);

```
void reset_timer(uint8_t timer_num);
```

Purpose: Reset and stop the selected timer using the familiar course name.

Parameters:

Declaration	Meaning
uint8_t timer_num	Hardware timer number 0 through 3.

Returns: No return value. Observe the documented hardware or shared-state effect.

Setup and ownership: Use timer 0 through 3. Configure its clock, prescaler and mode before starting when the question grades those values.

Hardware/shared side effects: May power the selected timer and change PCLKSEL, TCR, TC, PC, PR, CTCR, MCR, CCR, EMR, MR, IR and NVIC state.

Copyable example:

```
reset_timer(0);
```

Common mistake: Check PCLK, PR+1, units, timer number and W1C IR clearing. Never combine a callback helper with exact ownership of the same TIMERN vector.

RIT

uint32_t init_RIT(uint32_t interval);

```
uint32_t init_RIT(uint32_t interval);
```

Purpose: Initialize the RIT using the selected compile-time mode and the course return convention.

Parameters:

Declaration	Meaning
uint32_t interval	Direct-mode compare value; ignored by scheduler mode.

Returns: Uses the professor's convention: returns 0 on success and 1 on failure.

Setup and ownership: Scheduler and direct RIT modes are compile-time exclusive. Exact RIT ownership also excludes the scheduler callback path.

Hardware/shared side effects: May power RIT and change PCLKSEL1, RICOMPVAL, RIMASK, RICOUNTER, RICTRL and NVIC state.

Copyable example:

```
if (init_RIT(250000) == 0) enable_RIT();
```

Common mistake: Do not mix scheduler mode, direct RIT mode and an exact RIT handler. Only one model can own the RIT vector.

void enable_RIT(void);

```
void enable_RIT(void);
```

Purpose: Start the scheduler or direct RIT according to the selected compile-time mode.

Parameters: None.

Returns: No return value. Observe the documented hardware or shared-state effect.

Setup and ownership: Scheduler and direct RIT modes are compile-time exclusive. Exact RIT ownership also excludes the scheduler callback path.

Hardware/shared side effects: May power RIT and change PCLKSEL1, RICOMPVAL, RIMASK, RICOUNTER, RICTRL and NVIC state.

Copyable example:

```
enable_RIT();
```

Common mistake: Do not mix scheduler mode, direct RIT mode and an exact RIT handler. Only one model can own the RIT vector.

void disable_RIT(void);

```
void disable_RIT(void);
```

Purpose: Stop RIT counting.

Parameters: None.

Returns: No return value. Observe the documented hardware or shared-state effect.

Setup and ownership: Scheduler and direct RIT modes are compile-time exclusive. Exact RIT ownership also excludes the scheduler callback path.

Hardware/shared side effects: May power RIT and change PCLKSEL1, RICOMPVAL, RIMASK, RICOUNTER, RICTRL and NVIC state.

Copyable example:

```
disable_RIT();
```

Common mistake: Do not mix scheduler mode, direct RIT mode and an exact RIT handler. Only one model can own the RIT vector.

void reset_RIT(void);

```
void reset_RIT(void);
```

Purpose: Write zero to RICOUNTER.

Parameters: None.

Returns: No return value. Observe the documented hardware or shared-state effect.

Setup and ownership: Scheduler and direct RIT modes are compile-time exclusive. Exact RIT ownership also excludes the scheduler callback path.

Hardware/shared side effects: May power RIT and change PCLKSEL1, RICOMPVAL, RIMASK, RICOUNTER, RICTRL and NVIC state.

Copyable example:

```
reset_RIT();
```

Common mistake: Do not mix scheduler mode, direct RIT mode and an exact RIT handler. Only one model can own the RIT vector.

ADC and potentiometer

void ADC_init(void);

```
void ADC_init(void);
```

Purpose: Initialize ADC channel 5 with the normal course peripheral clock assumption.

Parameters: None.

Returns: No return value. Observe the documented hardware or shared-state effect.

Setup and ownership: Power/configure the ADC before conversions. If you own ADC_IRQHandler, read ADGDR once and decode/store the result yourself.

Hardware/shared side effects: May power ADC, change pin selection and ADCR/ADINTEN, start conversions, and consume ISR-cached results.

Copyable example:

```
ADC_init();
```

Common mistake: Fit JP12. ADGDR is read once by the built-in handler; if you own ADC_IRQHandler, read it once yourself and do not use cached helpers.

void ADC_start_conversion(void);

```
void ADC_start_conversion(void);
```

Purpose: Request one software ADC conversion using the familiar course name.

Parameters: None.

Returns: No return value. Observe the documented hardware or shared-state effect.

Setup and ownership: Power/configure the ADC before conversions. If you own ADC_IRQHandler, read ADGDR once and decode/store the result yourself.

Hardware/shared side effects: May power ADC, change pin selection and ADCR/ADINTEN, start conversions, and consume ISR-cached results.

Copyable example:

```
ADC_start_conversion();
```

Common mistake: Fit JP12. ADGDR is read once by the built-in handler; if you own ADC_IRQHandler, read it once yourself and do not use cached helpers.

Exact handlers and Reset_Handler

Exact timer handler

Enable the matching TIMER ownership switch. Read IR once, write the pending bits back because they are write-one-to-clear, perform the short required action, and return.

```
void TIMER0_IRQHandler(void)
{
    uint32_t pending = LPC_TIM0->IR;
    LPC_TIM0->IR = pending;
    if (pending & 1u) { /* short MR0 action */ }
}
```

Exact external-button handler

Enable the matching EINT ownership switch, call exam_button_irq_start once, and clear the corresponding EXTINT bit in the handler.

```
void EINT0_IRQHandler(void)
{
    LPC_SC->EXTINT = 1u;
    /* short required action */
}
```

Exact ADC handler

Read LPC_ADC->ADGDR exactly once, extract bits 15:4, and store the result in a volatile shared variable. Cached ADC helpers return BUSY while this handler is owned.

Reset_Handler

The startup file remains in the project and its reset handler is weak. Define a strong Reset_Handler in exam_asm.s only when requested. Perform reset-safe assembly work, then branch to __main. Do not return with BX LR and do not call C helpers before the runtime is initialized.

```
Reset_Handler    EXPORT Reset_Handler
                  IMPORT __main
                  PROC
                  ; reset-safe assembly work
                  LDR    R0, =__main
                  BX     R0
                  ENDP
```

Alphabetical function index

Function	Layer	Topic
adc_clock_hz	Precise driver layer	ADC and potentiometer
adc_configure_trigger	Precise driver layer	ADC and potentiometer
adc_init	Precise driver layer	ADC and potentiometer
ADC_init	Course-compatible layer	ADC and potentiometer
adc_read_channel	Precise driver layer	ADC and potentiometer
adc_read_value	Precise driver layer	ADC and potentiometer
adc_start_burst	Precise driver layer	ADC and potentiometer
adc_start_conversion	Precise driver layer	ADC and potentiometer
ADC_start_conversion	Course-compatible layer	ADC and potentiometer
adc_stop_burst	Precise driver layer	ADC and potentiometer
board_direct_register	Precise driver layer	Convenience and control
board_idle	Precise driver layer	Program control
board_init	Precise driver layer	Program control
board_self_test_run	Precise driver layer	Faults, SVC and hooks
BUTTON_init	Course-compatible layer	External buttons
button_irq_start	Precise driver layer	External buttons
button_is_pressed	Precise driver layer	External buttons
buttons_init	Precise driver layer	External buttons
buttons_pressed_mask	Precise driver layer	External buttons
buttons_set_confirmation_ms	Precise driver layer	External buttons
critical_enter	Precise driver layer	Interrupt communication
critical_exit	Precise driver layer	Interrupt communication
dac_init	Precise driver layer	DAC and analog output
dac_play_samples	Precise driver layer	DAC and analog output
dac_silence	Precise driver layer	DAC and analog output
dac_validate_update_rate	Precise driver layer	DAC and analog output
dac_write	Precise driver layer	DAC and analog output
disable_RIT	Course-compatible layer	RIT
disable_timer	Course-compatible layer	Timers
enable_RIT	Course-compatible layer	RIT
enable_timer	Course-compatible layer	Timers
event_flags_set	Precise driver layer	Interrupt communication
event_flags_take	Precise driver layer	Interrupt communication
exam_adc_read	Recommended exam layer	ADC and potentiometer
exam_button_event	Functions you write	Answer structure and callbacks
exam_button_irq_start	Recommended exam layer	External buttons
exam_button_pressed	Recommended exam layer	External buttons
exam_buttons_confirmation_ms	Recommended exam layer	External buttons
exam_buttons_pressed	Recommended exam layer	External buttons
exam_buttons_start	Recommended exam layer	External buttons

Function	Layer	Topic
exam_critical_enter	Recommended exam layer	Interrupt communication
exam_critical_exit	Recommended exam layer	Interrupt communication
exam_dac_percent	Recommended exam layer	DAC and analog output
exam_dac_silence	Recommended exam layer	DAC and analog output
exam_dac_write	Recommended exam layer	DAC and analog output
exam_events_set	Recommended exam layer	Interrupt communication
exam_events_take	Recommended exam layer	Interrupt communication
exam_idle	Recommended exam layer	Program control
exam_init	Recommended exam layer	Program control
exam_joystick_event	Functions you write	Answer structure and callbacks
exam_joystick_first	Recommended exam layer	Joystick
exam_joystick_read	Recommended exam layer	Joystick
exam_joystick_reset_first	Recommended exam layer	Joystick
exam_joystick_start	Recommended exam layer	Joystick
exam_led_off	Recommended exam layer	LED output
exam_led_on	Recommended exam layer	LED output
exam_led_read	Recommended exam layer	LED output
exam_led_toggle	Recommended exam layer	LED output
exam_led_write	Recommended exam layer	LED output
exam_leds_off	Recommended exam layer	LED output
exam_pot_read	Recommended exam layer	ADC and potentiometer
exam_pot_start	Recommended exam layer	ADC and potentiometer
exam_rit_start	Recommended exam layer	RIT
exam_rit_stop	Recommended exam layer	RIT
exam_rit_ticks	Recommended exam layer	RIT
exam_self_test	Recommended exam layer	Faults, SVC and hooks
exam_systick_every_ms	Recommended exam layer	SysTick
exam_systick_ticks	Recommended exam layer	SysTick
exam_timer_capture_happened	Recommended exam layer	Timers
exam_timer_clock_divider	Recommended exam layer	Timers
exam_timer_count	Recommended exam layer	Timers
exam_timer_event	Functions you write	Answer structure and callbacks
exam_timer_every_hz	Recommended exam layer	Timers
exam_timer_every_ms	Recommended exam layer	Timers
exam_timer_match	Recommended exam layer	Timers
exam_timer_match_happened	Recommended exam layer	Timers
exam_timer_prescaler	Recommended exam layer	Timers
exam_timer_reset	Recommended exam layer	Timers
exam_timer_start	Recommended exam layer	Timers
exam_timer_stop	Recommended exam layer	Timers
exam_user_10ms_hook	Precise driver layer	Faults, SVC and hooks

Function	Layer	Topic
exam_user_init	Functions you write	Answer structure and callbacks
exam_user_loop	Functions you write	Answer structure and callbacks
fault_traps_configure	Precise driver layer	Faults, SVC and hooks
init_RIT	Course-compatible layer	RIT
init_timer	Course-compatible layer	Timers
int0_pressed	Precise driver layer	External buttons
joystick_button_pressed	Precise driver layer	Joystick
joystick_down_pressed	Precise driver layer	Joystick
joystick_first_movement	Precise driver layer	Joystick
joystick_init	Precise driver layer	Joystick
joystick_is_pressed	Precise driver layer	Joystick
joystick_left_pressed	Precise driver layer	Joystick
joystick_read	Precise driver layer	Joystick
joystick_reset_first_movement	Precise driver layer	Joystick
joystick_right_pressed	Precise driver layer	Joystick
joystick_up_pressed	Precise driver layer	Joystick
key1_pressed	Precise driver layer	External buttons
key2_pressed	Precise driver layer	External buttons
led_all_off	Precise driver layer	LED output
LED_deinit	Course-compatible layer	LED output
LED_init	Course-compatible layer	LED output
led_off	Precise driver layer	LED output
LED_Off	Course-compatible layer	LED output
led_on	Precise driver layer	LED output
LED_On	Course-compatible layer	LED output
LED_Out	Course-compatible layer	LED output
led_read_mask	Precise driver layer	LED output
led_toggle	Precise driver layer	LED output
led_write8	Precise driver layer	LED output
led_write_mask	Precise driver layer	LED output
led_write_number	Precise driver layer	LED output
potentiometer_read	Precise driver layer	ADC and potentiometer
potentiometer_start	Precise driver layer	ADC and potentiometer
reset_RIT	Course-compatible layer	RIT
reset_timer	Course-compatible layer	Timers
rit_raw_configure	Precise driver layer	RIT
rit_raw_read	Precise driver layer	RIT
rit_raw_start	Precise driver layer	RIT
rit_raw_stop	Precise driver layer	RIT
rit_scheduler_start	Precise driver layer	RIT
rit_scheduler_stop	Precise driver layer	RIT

Function	Layer	Topic
rit_scheduler_ticks	Precise driver layer	RIT
speaker_write	Precise driver layer	DAC and analog output
speaker_write_percent	Precise driver layer	DAC and analog output
svc_dispatch	Precise driver layer	Faults, SVC and hooks
systick_calibration_value	Precise driver layer	SysTick
systick_configure	Precise driver layer	SysTick
systick_every_ms	Precise driver layer	SysTick
systick_has_precise_calibration	Precise driver layer	SysTick
systick_set_clock_source	Precise driver layer	SysTick
systick_start_periodic_ms	Precise driver layer	SysTick
systick_ticks	Precise driver layer	SysTick
timer_calculate_match_for_frequency	Precise driver layer	Timers
timer_capture_occurred	Precise driver layer	Timers
timer_configure_capture	Precise driver layer	Timers
timer_configure_external_match	Precise driver layer	Timers
timer_configure_frequency	Precise driver layer	Timers
timer_configure_match	Precise driver layer	Timers
timer_counter_clock_hz	Precise driver layer	Timers
timer_every_hz	Precise driver layer	Timers
timer_every_ms	Precise driver layer	Timers
timer_match_occurred	Precise driver layer	Timers
timer_peripheral_clock_hz	Precise driver layer	Timers
timer_read_capture	Precise driver layer	Timers
timer_read_counter	Precise driver layer	Timers
timer_reset	Precise driver layer	Timers
timer_set_callback	Precise driver layer	Timers
timer_set_clock_divider	Precise driver layer	Timers
timer_set_counter_mode	Precise driver layer	Timers
timer_set_prescaler	Precise driver layer	Timers
timer_start	Precise driver layer	Timers
timer_start_periodic_interrupt_hz	Precise driver layer	Timers
timer_start_periodic_interrupt_ms	Precise driver layer	Timers
timer_stop	Precise driver layer	Timers